

**CENTRALE
LYON**

ÉCOLE CENTRALE LYON

RAPPORT

**Gaussian Processes Augmented with Neural
Networks: From Variational Inference to Deep
Kernel Learning**

Élèves :

Pierre JOLY

Adam EL MANOUZI

Enseignant :

Christophette BLANCHET

February 10, 2026

Contents

1	Introduction	5
1.1	Context and motivation	5
1.2	Problem statement	5
1.3	Report organization	5
2	Kriging	7
2.1	Principle	7
2.2	Gaussian Process Regression	11
2.3	Hyperparameter optimization	13
2.4	Limitations and extensions	14
3	Parametric Bayesian Modeling	15
3.1	Principle	15
3.2	Bayesian linear regression	15
3.3	The problem of exact inference	18
3.4	Markov Chain Monte Carlo methods	20
4	Variational Inference	22
4.1	Principle	22
4.2	Variational family	23
4.3	Optimization	23
4.4	Comparison between MCMC and variational inference	30
5	Sparsity and Gaussian Processes	32
5.1	Variational inference for GPs	32
5.2	Predictive distribution and variational approximation	32
5.3	Variational lower bound (ELBO)	34
5.4	Inducing point selection	41
5.5	Stochastic Variational Inference (SVI)	41
6	Deep Kernel Learning	43
6.1	The GP problem in high dimensions	43
6.2	Principle of Deep Kernel Learning	43
6.3	Mathematical formulation	43
6.4	Model optimization	45
6.5	Algorithmic formalization	45
6.6	Toward a scalable extension of DKL: SVDKL	49
7	Experiments and Results	50
7.1	Dataset: Olivetti Faces	50
7.2	Experimental protocol	50
7.3	Neural networks	51
7.4	Optimization	52
7.5	Automatic differentiation	54
7.6	Results	56
8	Conclusion	58

Notation

Sets and spaces

- $\mathbb{N}$: set of natural numbers.
- $\mathbb{Z}$: set of integers.
- $\mathbb{R}$: set of real numbers.
- $\mathbb{R}^n$: Euclidean space of dimension n .
- $\mathcal{X}$: input variable space.
- $\mathcal{Y}$: output variable space.
- X : matrix (typically $n \times d$) gathering the n input observations (each row of X is $x_i^\top$).
- Y : vector (dimension n) gathering output observations $(y_1, \dots, y_n)^\top$.

Common operations and functions

- $\|x\|$: Euclidean norm of a vector $x \in \mathbb{R}^n$.
- $\langle x, y \rangle$: inner product of $x, y \in \mathbb{R}^n$.
- $\det(A)$: determinant of matrix A .
- $\text{tr}(A)$: trace of matrix A .
- $f : \mathcal{X} \rightarrow \mathcal{Y}$: function from $\mathcal{X}$ to $\mathcal{Y}$.
- ∇f : gradient of function f .
- Δf : Laplacian of f .
- $O(g(n))$: Landau notation for asymptotic behavior.

Probability and statistics

- $\mathbb{E}[X]$: expectation of random variable X .
- $\mathbb{V}[X]$: variance of random variable X .
- $\text{Cov}[X]$: covariance of random variable X .
- $\mathbb{P}(A)$: probability of event A .
- $X \sim \mathcal{N}(\mu, \sigma^2)$: random variable X follows a normal distribution with mean μ and variance σ^2 .
- $p(\theta)$: prior distribution on parameter θ .

- $p(\theta | y)$: posterior distribution on parameter θ given observed data y .
- $p(y | \theta)$: likelihood of observations y given θ .
- $p(y)$: marginal likelihood (evidence) of observations y .

Bayesian and variational inference

- $\text{KL}(q \| p)$: Kullback-Leibler divergence between distributions q and p .
- $\mathcal{L}(q)$ or ELBO: (*Evidence Lower BOund*) lower bound on $\log p(y)$ used in variational inference.

$$\mathcal{L}(q) = \mathbb{E}_q[\log p(y, \theta)] - \mathbb{E}_q[\log q(\theta)] = \mathbb{E}_q[\log p(y | \theta)] - \text{KL}(q \| p).$$

- $q(\theta)$: variational distribution chosen to approximate posterior $p(\theta | y)$.
- λ : variational parameters describing $q_\lambda(\theta)$.
- $\widehat{\nabla}_\lambda \mathcal{L}$: Monte Carlo estimator of the ELBO gradient with respect to λ .
- S : number of Monte Carlo samples used for estimation.

Gaussian processes and inducing-point approximations

- $\{x_i, y_i\}_{i=1}^n$: n observations (inputs $x_i \in \mathbb{R}^d$, outputs $y_i \in \mathbb{R}$).
- GP (Gaussian Process): family of functions where any finite subset of values follows a multivariate normal distribution.
- $k(\mathbf{x}, \mathbf{x}')$: kernel function of the Gaussian process.
- K_{nn} : $n \times n$ covariance matrix between the n training points.
- m : number of inducing points (or pseudo-points) used in the sparse approximation.
- X_m : set (or matrix) of the m inducing points; f_m : Gaussian process values at inducing points X_m .
- K_{mm} : $m \times m$ covariance matrix between inducing points X_m .
- K_{nm}, K_{mn} : cross-covariance matrices between the n training points and the m inducing points ($K_{mn} = K_{nm}^\top$).
- $Q_{nn} = K_{nm} K_{mm}^{-1} K_{mn}$: rank- m approximation of matrix K_{nn} using the Nyström method.
- σ^2 : variance (or noise hyperparameter) assumed known or estimated (e.g., in Gaussian regression).
- $\phi(f_m)$: variational distribution for process values at inducing points (often Gaussian $\mathcal{N}(\mu, \Lambda)$).

Miscellaneous

- I : identity matrix (size specified by context).
- A^{-1} : inverse matrix of A (when A is invertible).
- (y_{n+1}, x_{n+1}) : new (out-of-sample) data for which a value is predicted or inferred.

1 Introduction

1.1 Context and motivation

Gaussian processes (GPs) are a class of powerful and flexible probabilistic tools used in machine learning for tasks such as regression, classification, and optimization. Their ability to model functions in a non-parametric way, while providing rigorous uncertainty quantification, makes them ideal candidates for many applications. Historically, GPs originate from *kriging*, a stochastic spatial interpolation method developed in geostatistics to estimate values at unsampled locations from nearby observations. In machine learning, GPs generalize this approach by defining a *prior* over the function space, updated through Bayes' theorem to obtain a posterior distribution conditioned on observed data.

However, despite their benefits, GPs exhibit significant limitations in modern settings involving large datasets or high-dimensional input spaces. Their main drawback lies in computational complexity: training a standard GP requires inverting an $n \times n$ covariance matrix, where n is the number of data points, resulting in $\mathcal{O}(n^3)$ complexity. This constraint makes GPs difficult to apply beyond a few thousand observations. Moreover, in high-dimensional spaces, defining a suitable kernel that can capture complex nonlinear structures becomes problematic, which limits their effectiveness.

In parallel, deep neural networks (DNNs) have shown exceptional ability to learn complex representations from raw data, especially in domains such as computer vision and natural language processing. However, standard DNNs produce deterministic predictions and do not provide systematic uncertainty quantification, which is a critical limitation for applications requiring high reliability, such as automated decision-making or scientific analysis.

To overcome these limitations, hybrid approaches combining the strengths of GPs and DNNs have emerged. Among them, *Deep Kernel Learning* (DKL) proposes using a neural network to transform input data into a latent space where a GP can be applied more effectively. This transformation leverages the DNN ability to capture complex nonlinear relationships while preserving the uncertainty quantification offered by GPs. However, even with DKL, computational cost remains high, requiring approximation techniques such as inducing points or variational inference to make these models *scalable*.

1.2 Problem statement

This report investigates the integration of neural networks with Gaussian processes through Deep Kernel Learning, with emphasis on variational inference techniques and sparse approximations to improve scalability and efficiency. The main objective is to develop methods that leverage the advantages of GPs in scenarios where data are abundant and complex, while benefiting from the latent-space representation capabilities of DNNs.

1.3 Report organization

This report is progressively structured to cover fundamental concepts, technical advances, and experimental results:

- **Section 2:** Kriging - Presentation of kriging principles as the historical foundation of GPs, with its variants and links to Gaussian process regression.
- **Section 3:** Parametric Bayesian modeling - Introduction to the foundations of Bayesian modeling, illustrated by Bayesian linear regression, as a precursor to GPs.
- **Section 4:** Variational inference - Exploration of posterior approximation methods for complex models, essential for GP scalability.
- **Section 5:** Sparsity and Gaussian processes - Discussion of *sparse* approximation techniques, especially through inducing points, to reduce computational complexity.
- **Section 6:** Deep Kernel Learning - Presentation of DKL, combining DNNs and GPs to improve high-dimensional modeling.
- **Section 7:** Experiments - Application of the discussed concepts on an image dataset, illustrating the effectiveness of hybrid models.
- **Section 8:** Conclusion.

2 Kriging

2.1 Principle

Kriging is a stochastic spatial interpolation method (Matheron, 1969). It is a linear estimator that uses spatial covariance to predict the value of a regional variable at an unsampled point in space D , from values observed at nearby points. The estimator is modeled by a random field Z . Kriging prediction corresponds to the best linear unbiased predictor (BLUP) of Z at unsampled locations, in the minimum-variance sense. The random process is decomposed as follows:

$$Z(x) = \mu(x) + \delta(x), \quad \forall x \in D,$$

where μ is deterministic and δ is a zero-mean random variable with a known dependence structure. The kriging model differs from the classical regression model because errors are assumed to be spatially dependent. This random process satisfies a form of stationarity.

Stationarity of Z

It is necessary to introduce a form of regularity to handle the problem. Three notions of regularity are used in kriging.

Strict stationarity: all characteristics of the probability distribution are assumed to be translation-invariant. Formally, for any h and any $x_i \in D$, the random vector $(Z(x_1), \dots, Z(x_n))$ has the same distribution as $(Z(x_1 + h), \dots, Z(x_n + h))$. This is a very restrictive assumption and is rarely used in practice.

Second-order stationarity, or weak stationarity: the first two moments are assumed to be translation-invariant. Formally:

- $\mu(x) = \mathbb{E}[Z(x)] = \mu$ is constant,
- $\text{Cov}(Z(x), Z(x'))$, the covariance depends only on the lag h between two points.

In particular, this implies that variance is constant and finite. Weak stationarity allows the definition of a global covariance model k . However, in many real situations, the mean may vary spatially or the variance may be unbounded over large domains.

Intrinsic stationarity: increments are assumed to have zero mean and finite variance that depends only on h . Formally,

- $\mathbb{E}[Z(x + h) - Z(x)] = 0$,
- $\mathbb{V}[Z(x + h) - Z(x)]$ exists, is finite, and depends only on h

These conditions allow the construction of the variogram function. Intrinsic stationarity is more general than second-order stationarity: any weakly stationary process is intrinsically stationary, but the converse is not always true.

Types of kriging

There are several kriging variants depending on the assumption made on μ .

Simple kriging: μ is assumed known and constant over the whole domain D . This variant is associated with weak stationarity.

Ordinary kriging: μ is assumed unknown but constant over the whole domain D . This variant is associated with weak stationarity.

Universal kriging: μ is assumed to be a linear combination of linearly independent functions, $\mu = \sum_{l=1}^p \beta_l f_l$. This variant is associated with intrinsic stationarity.

Kriging prediction

Let $x_* \in D$ be a point where we want to estimate $Z(x_*)$ from n available observations $Y = (Z(x_1), \dots, Z(x_n))^\top$ and $X = (x_1, \dots, x_n)^\top$. Kriging corresponds to a minimum-variance optimization over the class of linear estimators. Linear estimators have the form

$$\hat{Z}(x_*) = \sum_{j=1}^n \lambda_j Z(x_j) \cdot Y,$$

where λ_j are the weights to determine. The optimization is subject to the unbiasedness constraint, i.e., $\mathbb{E}[\hat{Z}(x_*)] = \mathbb{E}[Z(x_*)]$.

The optimization problem is therefore written as:

$$(\lambda_1, \dots, \lambda_n) = \arg \min_{(\lambda_1, \dots, \lambda_n) \in \mathbb{R}^n} \mathbb{V}[Z(x_*) - \hat{Z}(x_*)] \quad \text{subject to} \quad \mathbb{E}[\hat{Z}(x_*)] = \mathbb{E}[Z(x_*)].$$

In simple kriging, since the mean is known, it is enough to apply optimization to the centered estimator $Z_0(x) = Z(x) - \mu$ so that the constraint is satisfied. In ordinary kriging, the condition leads to

$$\begin{aligned} \mathbb{E}[Z(x_*)] &= \mathbb{E}[\hat{Z}(x_*)] \\ \iff \mu &= \sum_{j=1}^n \lambda_j \mu \end{aligned}$$

If the mean is zero then the constraint is satisfied; otherwise the constraint is equivalent to $\sum_{i=1}^n \lambda_i = 1$. More generally, in universal kriging,

$$\begin{aligned} \mathbb{E}[Z(x_*)] &= \mathbb{E}[\hat{Z}(x_*)] \\ \iff \sum_{l=1}^p \beta_l f_l(x_*) &= \sum_{j=1}^n \lambda_j \sum_{l=1}^p \beta_l f_l(x_j) \\ \iff f(x_*)^\top \beta &= \left(\sum_{j=1}^n \lambda_j f(x_j)^\top \right) \beta, \end{aligned}$$

with $f = (f_1 \cdots f_p)^\top$ and $\beta = (\beta_1 \cdots \beta_p)^\top$. Since this expression holds for all β ,

$$\begin{aligned} \mathbb{E}[Z(x_*)] &= \mathbb{E}[\hat{Z}(x_*)] \\ \iff f(x_*) &= \sum_{j=1}^n \lambda_j f(x_j) \\ \iff F(X)^\top \lambda &= f(x_*), \end{aligned}$$

with $F(X) = (f(x_1) \cdots f(x_n))_{n \times p}^\top$.

The method of Lagrange multipliers is applied. The Lagrangian is defined as

$$\mathcal{L}(\lambda, \gamma) = \mathbb{V}[Z(x_*) - \hat{Z}(x_*)] + \gamma^\top (F(X)^\top \lambda - f(x_*))$$

The system of equations is given by

$$\frac{\partial}{\partial \lambda} \mathcal{L}(\lambda, \gamma) = 0 \quad \wedge \quad \frac{\partial}{\partial \gamma} \mathcal{L}(\lambda, \gamma) = 0$$

Expanding the variance $\mathbb{V}[Z(x_*) - \hat{Z}(x_*)]$ gives

$$\begin{aligned} \mathbb{V}[Z(x_*) - \hat{Z}(x_*)] &= \mathbb{V}[Z(x_*)] + \mathbb{V}[\hat{Z}(x_*)] - 2 \text{Cov}[\hat{Z}(x_*), Z(x_*)] \\ &= \mathbb{V}[Z(x_*)] + \mathbb{V}\left[\sum_{j=1}^n \lambda_j Z(x_j)\right] - 2 \text{Cov}\left[\sum_{j=1}^n \lambda_j Z(x_j), Z(x_*)\right] \\ &= \mathbb{V}[Z(x_*)] + \sum_{j=1}^n \sum_{k=1}^n \lambda_j \lambda_k \text{Cov}[Z(x_j), Z(x_k)] - 2 \sum_{j=1}^n \lambda_j \text{Cov}[Z(x_j), Z(x_*)] \end{aligned}$$

Injecting this expression into the system gives

$$\begin{cases} \forall i \in [1, n], & -2 \text{Cov}[Z(x_*), Z(x_i)] + \sum_{k=1}^n \lambda_k \text{Cov}[Z(x_i), Z(x_k)] + \sum_{j=1}^n \lambda_j \text{Cov}[Z(x_j), Z(x_i)] \\ & + \gamma^\top \sum_{j=0}^p f(x_j) \lambda_i = 0 \\ F(X)^\top \lambda - f(x_*) = 0. \end{cases}$$

By covariance symmetry, we obtain

$$\forall i \in [1, n], \quad 2 \sum_{j=1}^n \lambda_j \text{Cov}[Z(x_j), Z(x_i)] + \gamma^\top \sum_{j=0}^p f(x_j) \lambda_i = 2 \text{Cov}[Z(x_*), Z(x_i)].$$

Let $K_{nn} = (\text{Cov}[Z(x_i), Z(x_j)])_{1 \leq i, j \leq n}$, $\lambda = (\lambda_i)_{1 \leq i \leq n}$, and $c = (\text{Cov}[Z(x_i), Z(x_*)])_{1 \leq i \leq n}$. The system is then equivalent to

$$\begin{pmatrix} K_{nn} & F(X) \\ F(X)^\top & 0 \end{pmatrix} \cdot \begin{pmatrix} \lambda \\ \gamma \end{pmatrix} = \begin{pmatrix} c \\ f(x_*) \end{pmatrix}$$

Thus,

$$\begin{cases} K_{nn} \lambda + F(X) \gamma &= c \\ F(X)^\top \lambda &= f(x_*) \end{cases}$$

Injecting the first equation into the second,

$$\begin{cases} \lambda &= K_{nn}^{-1} (c - F(X) \gamma) \\ F(X)^\top K_{nn}^{-1} (c - F(X) \gamma) &= f(x_*) \end{cases}$$

Then by rewriting,

$$\begin{cases} \lambda &= K_{nn}^{-1} (c - F(X) \gamma) \\ F(X)^\top K_{nn}^{-1} c - f(x_*) &= F(X)^\top K_{nn}^{-1} F(X) \gamma \end{cases}$$

Finally,

$$\begin{cases} \lambda &= K_{nn}^{-1} (c - F(X) \gamma) \\ \gamma &= (F(X)^\top K_{nn}^{-1} F(X))^{-1} [F(X)^\top K_{nn}^{-1} c - f(x_*)] \end{cases}$$

In summary,

$$\lambda = K_{nn}^{-1} (c - F(X) (F(X)^\top K_{nn}^{-1} F(X))^{-1} [F(X)^\top K_{nn}^{-1} c - f(x_*)])$$

The prediction is then given by

$$\hat{Z}(x_*) = f(x_*)^\top \hat{\beta} + c^\top K_{nn}^{-1} [Y - F(X) \hat{\beta}], \quad \text{with} \quad \hat{\beta} = (F(X)^\top K_{nn}^{-1} F(X))^{-1} F(X)^\top K_{nn}^{-1} Y.$$

By repeating the process, this formulation can be extended to estimating m target points $X_* = \{x_{*,1} \dots x_{*,m}\}$. Let

$$K_{mn} = \begin{pmatrix} c^{(1)} \\ \vdots \\ c^{(m)} \end{pmatrix} = \begin{pmatrix} \text{Cov}[Z(x_{*,1}), Z(x_1)] & \dots & \text{Cov}[Z(x_{*,1}), Z(x_n)] \\ \vdots & \ddots & \vdots \\ \text{Cov}[Z(x_{*,m}), Z(x_1)] & \dots & \text{Cov}[Z(x_{*,m}), Z(x_n)] \end{pmatrix}$$

Hence,

$$\hat{Z}(X_*) = F(X_*)^\top \hat{\beta} + K_{mn}^\top K_{nn}^{-1} [Y - F(X) \hat{\beta}], \quad \text{with} \quad \hat{\beta} = (F(X)^\top K_{nn}^{-1} F(X))^{-1} F(X)^\top K_{nn}^{-1} Y.$$

2.2 Gaussian Process Regression

Gaussian process (GP) regression is formulated in the kriging framework, where the random field is modeled by a Gaussian process. A Gaussian process is defined so that any finite collection $(Z(x_0), \dots, Z(x_n))$ of process elements follows a joint normal distribution,

$$(Z(x_0), \dots, Z(x_n)) \sim \mathcal{N}(\mu, K_{nn}),$$

with $\mu = \mathbb{E}[(Z(x_0), \dots, Z(x_n))]$ and $K_{nn} = (\text{Cov}(Z(x_i), Z(x_j)))_{1 \leq i, j \leq n}$.

The mean function $\mu(x)$ depends on the type of kriging considered. The intrinsic stationarity assumption imposes that covariance depends only on the lag between two points. Several covariance functions k are commonly used. We present several kernels.

Exponential kernel

The exponential kernel models a rapid decay of spatial dependence as a function of the distance between two points:

$$k_{\text{exp}}(x, x') = \sigma^2 \exp\left(-\frac{\|x - x'\|}{\ell}\right),$$

where σ^2 is the marginal variance of the process, and ℓ is the characteristic length scale controlling the spatial range of correlations. This kernel defines continuous but non-differentiable sample paths at zero.

Matérn kernel

The Matérn kernel generalizes the exponential kernel by introducing a parameter ν , called the smoothness parameter, which controls the regularity of process realizations:

$$k_{\text{Matérn}}(x, x') = \sigma^2 \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\sqrt{2\nu} \frac{\|x - x'\|}{\ell}\right)^\nu K_\nu \left(\sqrt{2\nu} \frac{\|x - x'\|}{\ell}\right),$$

where K_ν is the modified Bessel function of the second kind and order ν , Γ is the Gamma function, and $\nu > 0$. When $\nu \rightarrow \infty$, we recover the Gaussian kernel, and when $\nu = 1/2$, this kernel coincides with the exponential kernel. The Matérn kernel is widely used in practice thanks to its flexibility for modeling different smoothness levels.

Gaussian (RBF) kernel

The Gaussian kernel, also called the Radial Basis Function (RBF), is one of the most widely used kernels in practice because of its infinite smoothness:

$$k_{\text{RBF}}(x, x') = \sigma^2 \exp\left(-\frac{\|x - x'\|^2}{2\ell^2}\right).$$

This kernel generates infinitely differentiable sample paths and is particularly suitable when the studied phenomenon is very smooth.

Using the kriging formula again,

$$\hat{Z}(X_*) = F(X_*)^\top \hat{\beta} + K_{mn}^\top K_{nn}^{-1} [Y - F(X) \hat{\beta}], \quad \text{with} \quad \hat{\beta} = (F(X)^\top K_{nn}^{-1} F(X))^{-1} F(X)^\top K_{nn}^{-1} Y,$$

we can specify $K_{mn} = (k(x_{*,i}, (x_j)))_{1 \leq i \leq m, 1 \leq j \leq n}$ and $K_{nn} = (k(x_i, x_j))_{1 \leq i, j \leq n}$. In general, kernels have hyperparameters σ , the marginal process variance, and ℓ , the characteristic length scale.

Kernels vanish for identical points. Thus, matrix K_{nn} has a zero diagonal, which guarantees interpolation. As a consequence, variance is zero at measured points. However, measurements may be noisy. To modify this property, a diagonal term must be added. Since measurements are i.i.d., the same term must be added for each measurement. A third hyperparameter is then introduced, σ_n , the measurement noise, and σ_f denotes the process marginal variance. This parameter is added as follows,

$$K_{nn} \leftarrow K_{nn} + \sigma_n I_{nn}$$

The prediction produced by kriging provides a conditional Gaussian distribution at each test point X_* , written as:

$$Z(X_*) \mid X, Y \sim \mathcal{N}(\mu(X_*), \Sigma(X_*)),$$

where,

- the conditional mean is given by:

$$\mu(X_*) = F(X_*)^\top \hat{\beta} + K_{mn}^\top K_{nn}^{-1} (Y - F(X) \hat{\beta}),$$

- the conditional covariance matrix between test points is:

$$\Sigma(X_*) = K_{mm} - K_{mn}^\top K_{nn}^{-1} K_{mn} + (F(X_*) - K_{mn}^\top K_{nn}^{-1} F(X))^\top (F(X)^\top K_{nn}^{-1} F(X))^{-1} (F(X) - K_{mn}^\top K_{nn}^{-1} F(X_*))$$

In particular, the predictive variance at a point x_* is given by:

$$\mathbb{V}[Z(x_*) \mid X, Y] = k(x_*, x_*) - k(x_*, X)^\top K_{nn}^{-1} k(x_*, X) + r(x_*)^\top (F^\top K_{nn}^{-1} F)^{-1} r(x_*),$$

$$\text{with } r(x_*) = F(x_*) - F^\top K_{nn}^{-1} k(x_*, X).$$

GP regression is solved for a given set of hyperparameters $\theta = \{\sigma_f, \ell, \sigma_n\}$. However, these parameters strongly affect prediction quality and their optimal values depend on the data. It is therefore necessary to optimize these hyperparameters.

2.3 Hyperparameter optimization

The quality of Gaussian process prediction strongly depends on the choice of kernel hyperparameters $\theta = \{\sigma_f, \ell, \sigma_n\}$. These hyperparameters determine the shape and smoothness of realizable functions, as well as the relative importance of measurement noise. They are generally optimized by maximizing the log marginal likelihood. Observations follow a normal distribution,

$$Y = Z(X) \sim \mathcal{N}(\mu(X), K_{nn}(\theta)).$$

Thus the likelihood is given by

$$p(Y | X, \theta) = \frac{1}{(2\pi)^{n/2} |K_{nn}(\theta)|^{1/2}} \exp \left(-\frac{1}{2} (Y - \mu(X))^T K_{nn}^{-1}(\theta) (Y - \mu(X)) \right).$$

Finally, we obtain

$$\log p(Y | X, \theta) = -\frac{1}{2} Y^T K_{nn}^{-1}(\theta) Y - \frac{1}{2} \log |K_{nn}(\theta)| - \frac{n}{2} \log(2\pi).$$

In practice, maximization is often performed using gradient-based methods, notably L-BFGS. L-BFGS is a gradient method that uses a Hessian approximation to accelerate convergence.

Gradient methods require computing partial derivatives. The derivative of the log marginal likelihood with respect to a hyperparameter θ_j is computed as follows:

$$\frac{\partial}{\partial \theta_j} \log p(Y | X, \theta) = \frac{\partial}{\partial \theta_j} \left(-\frac{1}{2} Y^T K_{nn}^{-1} Y - \frac{1}{2} \log |K_{nn}| - \frac{n}{2} \log(2\pi) \right).$$

The term $\frac{n}{2} \log(2\pi)$ is constant and disappears under differentiation:

$$\frac{\partial}{\partial \theta_j} \log p(Y | X, \theta) = -\frac{1}{2} \frac{\partial}{\partial \theta_j} (Y^T K_{nn}^{-1} Y) - \frac{1}{2} \frac{\partial}{\partial \theta_j} (\log |K_{nn}|).$$

Each term is computed separately. Since $\frac{\partial K_{nn}^{-1}}{\partial \theta_j} = -K_{nn}^{-1} \frac{\partial K_{nn}}{\partial \theta_j} K_{nn}^{-1}$, we get:

$$-\frac{1}{2} \frac{\partial}{\partial \theta_j} (Y^T K_{nn}^{-1} Y) = -\frac{1}{2} Y^T \left(-K_{nn}^{-1} \frac{\partial K_{nn}}{\partial \theta_j} K_{nn}^{-1} \right) Y = \frac{1}{2} Y^T K_{nn}^{-1} \frac{\partial K_{nn}}{\partial \theta_j} K_{nn}^{-1} Y.$$

Setting $\alpha = K_{nn}^{-1} Y$, we obtain:

$$= \frac{1}{2} \alpha^T \frac{\partial K_{nn}}{\partial \theta_j} \alpha.$$

The derivative of the log-determinant is:

$$-\frac{1}{2} \frac{\partial}{\partial \theta_j} \log |K_{nn}| = -\frac{1}{2} \text{tr} \left(K_{nn}^{-1} \frac{\partial K_{nn}}{\partial \theta_j} \right).$$

We therefore obtain:

$$\frac{\partial}{\partial \theta_j} \log p(Y|X, \theta) = \frac{1}{2} \alpha^\top \frac{\partial K_{nn}}{\partial \theta_j} \alpha - \frac{1}{2} \text{tr} \left(K_{nn}^{-1} \frac{\partial K_{nn}}{\partial \theta_j} \right).$$

Noting that $\alpha^\top \frac{\partial K_{nn}}{\partial \theta_j} \alpha$ is a scalar, equal to its own trace,

$$\frac{\partial}{\partial \theta_j} \log p(Y|X, \theta) = \frac{1}{2} \left(\text{tr} \left(\alpha \alpha^\top \frac{\partial K_{nn}}{\partial \theta_j} \right) - \text{tr} \left(K_{nn}^{-1} \frac{\partial K_{nn}}{\partial \theta_j} \right) \right).$$

Finally, by factoring into a single trace:

$$\frac{\partial}{\partial \theta_j} \log p(Y|X, \theta) = \frac{1}{2} \text{tr} \left[(\alpha \alpha^\top - K_{nn}^{-1}) \frac{\partial K_{nn}}{\partial \theta_j} \right],$$

recalling that $\alpha = K_{nn}^{-1} Y$, we finally obtain:

$$\frac{\partial}{\partial \theta_j} \log p(Y|X, \theta) = \frac{1}{2} \text{tr} \left[(K_{nn}^{-1} Y Y^\top K_{nn}^{-1} - K_{nn}^{-1}) \frac{\partial K_{nn}}{\partial \theta_j} \right].$$

2.4 Limitations and extensions

Although very flexible and interpretable, Gaussian process regression has high algorithmic complexity, on the order of $\mathcal{O}(n^3)$. This complexity mainly comes from inverting the $n \times n$ covariance matrix K_{nn} . This limitation makes the classical approach impractical once dataset size exceeds a few thousand points. Methods have been developed to overcome these limitations.

3 Parametric Bayesian Modeling

3.1 Principle

In parametric Bayesian modeling, model parameters are treated as random variables (Laplace, 1820; Box and Tiao, 2011). A *prior* distribution $p(\theta)$ is assigned to parameters θ . Given observations $\{(x_i, y_i)\}_{i=1}^n$, the goal is to estimate the *posterior* distribution

$$p(\theta | y),$$

where $y = \{y_i, x_i\}_{i=1}^n$ denotes the set of observed data, assumed to be independent realizations from the underlying joint distribution $(\mathcal{Y}, \mathcal{X})$. This distribution allows inference on new data by integrating uncertainty over parameters, according to:

$$p(y_{n+1} | x_{n+1}, y) = \int p(y | x_{n+1}, \theta) p(\theta | y) d\theta.$$

Applying Bayes' theorem leads to:

$$p(\theta | y) = \frac{p(y | \theta) p(\theta)}{p(y)},$$

where $p(y | \theta)$ is the model likelihood and the evidence $p(y)$, also called the marginal likelihood, is obtained by marginalization,

$$p(y) = \int p(y | \theta) p(\theta) d\theta.$$

3.2 Bayesian linear regression

A classic example is Bayesian linear regression under the assumption of normally distributed residuals.

A set of n observations $y = \{(x_i, y_i)\}_{i=1}^n$ is considered, with $x_i \in \mathbb{R}^d$ and $y_i \in \mathbb{R}$, generated by the linear regression model

$$y_i = x_i^T \theta + \varepsilon_i,$$

with $\theta \in \mathbb{R}^d$ the parameter vector, and $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$ i.i.d. Gaussian noise terms. To simplify calculations, variance σ^2 is assumed known in this example. In the classical setting, a prior is assigned to it using an inverse-gamma distribution.

The Bayesian assumptions are as follows:

- A Gaussian prior on θ :

$$p(\theta) = \mathcal{N}(\theta | \mu_\theta, \Sigma_\theta),$$

where μ_θ and Σ_θ are prior parameters.

- A Gaussian likelihood conditional on θ :

$$p(y | \theta) = \prod_{i=1}^n \mathcal{N}(y_i | x_i^T \theta, \sigma^2).$$

The following relation, introduced above, is used to infer y_{n+1} from a new input x_{n+1} :

$$p(y_{n+1} | x_{n+1}, y) = \int p(y_{n+1} | x_{n+1}, \theta) p(\theta | y) d\theta.$$

The posterior distribution is computed using Bayes' theorem:

$$p(\theta | y) = \frac{p(y | \theta) p(\theta)}{p(y)},$$

The posterior is then proportional to:

$$p(\theta | y) \propto p(y | \theta) p(\theta) \propto \exp \left[-\frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - x_i^\top \theta)^2 - \frac{1}{2} (\theta - \mu_\theta)^\top \Sigma_\theta^{-1} (\theta - \mu_\theta) \right].$$

Introducing matrix X of size $n \times d$, whose i -th row is $x_i^\top$, and vector $Y = (y_1, \dots, y_n)^\top$, we get:

$$\sum_{i=1}^n (y_i - x_i^\top \theta)^2 = \|Y - X\theta\|^2.$$

The density becomes:

$$p(\theta | y) \propto \exp \left[-\frac{1}{2} \left\{ \frac{1}{\sigma^2} (Y - X\theta)^\top (Y - X\theta) + (\theta - \mu_\theta)^\top \Sigma_\theta^{-1} (\theta - \mu_\theta) \right\} \right].$$

The objective is to rewrite the exponent as a completed square in θ , i.e. in the form:

$$-\frac{1}{2} (\theta - \mu)^\top \Sigma (\theta - \mu) + \text{constant},$$

To do so, the quadratic forms are expanded separately.

The prior-related part is:

$$(\theta - \mu_\theta)^\top \Sigma_\theta^{-1} (\theta - \mu_\theta) = \theta^\top \Sigma_\theta^{-1} \theta - 2\mu_\theta^\top \Sigma_\theta^{-1} \theta + \mu_\theta^\top \Sigma_\theta^{-1} \mu_\theta.$$

The likelihood-related part is:

$$\frac{1}{\sigma^2} (Y - X\theta)^\top (Y - X\theta) = \frac{1}{\sigma^2} (Y^\top Y - 2Y^\top X\theta + \theta^\top X^\top X\theta).$$

Combining both expansions, the exponent becomes:

$$-\frac{1}{2} \left\{ \theta^\top \left(\frac{1}{\sigma^2} X^\top X + \Sigma_\theta^{-1} \right) \theta - 2\theta^\top \left(\frac{1}{\sigma^2} X^\top Y + \Sigma_\theta^{-1} \mu_\theta \right) + \frac{1}{\sigma^2} Y^\top Y + \mu_\theta^\top \Sigma_\theta^{-1} \mu_\theta \right\}.$$

For simpler notation, define:

$$A = \frac{1}{\sigma^2} X^\top X + \Sigma_\theta^{-1}, \quad b = \frac{1}{\sigma^2} X^\top Y + \Sigma_\theta^{-1} \mu_\theta, \quad \text{et} \quad c = \frac{1}{\sigma^2} Y^\top Y + \mu_\theta^\top \Sigma_\theta^{-1} \mu_\theta.$$

The exponent can then be written as:

$$-\frac{1}{2} \{ \theta^\top A \theta - 2 \theta^\top b + c \}.$$

A is symmetric and positive definite by construction, therefore:

$$\theta^\top A \theta - 2 \theta^\top b = (\theta - \mu)^\top A (\theta - \mu) - \mu^\top A \mu,$$

with mean vector μ given by

$$\mu = A^{-1}b = \left(\frac{1}{\sigma^2} X^\top X + \Sigma_\theta^{-1} \right)^{-1} \left(\frac{1}{\sigma^2} X^\top Y + \Sigma_\theta^{-1} \mu_\theta \right).$$

Substituting into the exponent gives:

$$-\frac{1}{2} \{ (\theta - \mu)^\top A (\theta - \mu) - \mu^\top A \mu + c \}.$$

Terms that do not depend on θ , namely $-\mu^\top A \mu + c$, can be absorbed into the normalization constant. It follows that the posterior density is:

$$p(\theta | y) \propto \exp \left[-\frac{1}{2} (\theta - \mu)^\top A (\theta - \mu) \right].$$

The resulting expression corresponds to the density of a multivariate normal distribution with mean vector μ and covariance matrix $\Sigma = A^{-1}$, that is:

$$\theta | y \sim \mathcal{N}(\mu, \Sigma) \quad \text{where} \quad \Sigma = \left(\frac{1}{\sigma^2} X^\top X + \Sigma_\theta^{-1} \right)^{-1}.$$

To predict a new observation y_{n+1} associated with input x_{n+1} , we use the predictive distribution:

$$p(y_{n+1} | x_{n+1}, y) = \int p(y_{n+1} | x_{n+1}, \theta) p(\theta | y) d\theta,$$

The term $p(y_{n+1} | x_{n+1}, \theta)$ is:

$$p(y_{n+1} | x_{n+1}, \theta) = \mathcal{N}(x_{n+1}^\top \theta, \sigma^2)$$

The goal is to show that this integral is the convolution of two Gaussian distributions. The first step is the change of variable $z = x_{n+1}^\top \theta$. An orthonormal basis $\{v_i\}_{i=1}^d$ is built such that its first vector v_1 is parallel to x_{n+1} , $v_1 = \frac{x_{n+1}}{\|x_{n+1}\|}$, and the others are orthogonal to it. The parameter vector θ has a unique decomposition in this basis.

$$\theta = \sum_{i=1}^d u_i v_i, \quad u_i = v_i^\top \theta, \quad u_1 = \frac{z}{\|x_{n+1}\|}$$

The rotation matrix associated with the basis change between the canonical basis and $\{v_i\}_{i=1}^d$ is denoted V . Define

$$\tilde{\mu} = V^\top \mu, \quad \tilde{\Sigma} = V^\top \Sigma V$$

This change of basis gives:

$$\begin{aligned}\mathcal{N}(\theta \mid \mu, \Sigma) &= \mathcal{N}\left(\sum_{i=1}^d u_i v_i \mid \mu, \Sigma\right) \\ &= \mathcal{N}(u \mid \tilde{\mu}, \tilde{\Sigma})\end{aligned}$$

The transformation between θ and u is linear and orthonormal, so the Jacobian determinant is 1. Thus, using this change of variable and Fubini's theorem, we obtain:

$$\begin{aligned}p(y_{n+1} \mid x_{n+1}, y) &= \int \dots \int \mathcal{N}(y_{n+1} \mid u_1 \|x_{n+1}\|, \sigma^2) \cdot \mathcal{N}(u \mid \tilde{\mu}, \tilde{\Sigma}) du_1 \dots du_d \\ &= \int \mathcal{N}(y_{n+1} \mid u_1 \|x_{n+1}\|, \sigma^2) \left(\int \dots \int \mathcal{N}(u \mid \tilde{\mu}, \tilde{\Sigma}) du_2 \dots du_d \right) du_1\end{aligned}$$

By marginalizing the normal distribution, we get:

$$\begin{aligned}p(y_{n+1} \mid x_{n+1}, y) &= \int \mathcal{N}(y_{n+1} \mid u_1 \|x_{n+1}\|, \sigma^2) \mathcal{N}(u_1 \mid \tilde{\mu}_1, \tilde{\Sigma}_{11}) du_1 \\ &= \int \mathcal{N}(y_{n+1} \mid z, \sigma^2) \mathcal{N}(z \mid \|x_{n+1}\| \tilde{\mu}_1, \|x_{n+1}\|^2 \tilde{\Sigma}_{11}) dz \\ &= \frac{1}{\sqrt{4\pi^2 \sigma^2 \|x_{n+1}\|^2 \tilde{\Sigma}_{11}}} \int \exp \left[-\frac{(y_{n+1} - z)^2}{2\sigma^2} \right] \exp \left[-\frac{(z - \|x_{n+1}\| \tilde{\mu}_1)^2}{2\|x_{n+1}\|^2 \tilde{\Sigma}_{11}} \right] dz \\ &= \frac{1}{\sqrt{4\pi^2 \sigma^2 \|x_{n+1}\|^2 \tilde{\Sigma}_{11}}} \int \exp \left[-\frac{(y_{n+1} - z)^2}{2\sigma^2} - \frac{(z - \|x_{n+1}\| \tilde{\mu}_1)^2}{2\|x_{n+1}\|^2 \tilde{\Sigma}_{11}} \right] dz\end{aligned}$$

It remains to expand the quadratic terms and complete the square as before. We finally obtain:

$$p(y_{n+1} \mid x_{n+1}, y) = \mathcal{N}\left(y_{n+1} \mid \|x_{n+1}\| \tilde{\mu}_1, \sigma^2 + \|x_{n+1}\|^2 \tilde{\Sigma}_{11}\right).$$

This example illustrates a particular conjugate-family case, where a Gaussian prior and Gaussian likelihood yield a Gaussian posterior. It shows that exact Bayesian inference is feasible in this restricted setting. However, even in this simple case, explicit computation of $p(y)$ was avoided. Thus, outside such conjugate families, exact Bayesian inference generally becomes impossible or very difficult without approximate methods.

3.3 The problem of exact inference

Bayesian inference relies on Bayes' theorem, which expresses the posterior distribution of model parameters as:

$$p(\theta \mid y) = \frac{p(y \mid \theta) p(\theta)}{p(y)}.$$

In this expression, the evidence term plays a crucial role because it normalizes the posterior distribution. However, in most complex Bayesian models, its analytical computation is impossible. Therefore, explicit normalization of the posterior is generally intractable.

Example of a model where exact inference is impossible: Gaussian mixture

The impossibility of computing evidence can be illustrated with a Bayesian Gaussian Mixture Model (BGMM) (Blei et al., 2017). Consider a Bayesian mixture of univariate Gaussian components with unit variance, with K mixture components corresponding to K Gaussian distributions with means $\mu = \{\mu_1, \dots, \mu_K\}$. Mean parameters are assumed independently drawn from a shared prior $p(\mu_k)$, chosen as a Gaussian distribution $\mathcal{N}(0, \sigma^2)$, where σ^2 is a hyperparameter. Generating an observation x_i in this model follows several steps. First, a cluster assignment c_i is sampled, indicating which latent cluster x_i belongs to. This assignment is represented by a K -dimensional indicator vector with zeros everywhere except at the assigned cluster position. Then, observation x_i is sampled from the corresponding Gaussian $\mathcal{N}(c_i^\top \mu, 1)$.

The full hierarchical model is defined by:

$$\mu_k \sim \mathcal{N}(0, \sigma^2), \quad k = 1, \dots, K$$

$$c_i \sim \text{Categorical}(1/K, \dots, 1/K), \quad i = 1, \dots, n$$

$$x_i | c_i, \mu \sim \mathcal{N}(c_i^\top \mu, 1), \quad i = 1, \dots, n$$

For a sample of size n , the joint density of latent and observed variables is:

$$p(\mu, c, x) = p(\mu) \prod_{i=1}^n p(c_i) p(x_i | c_i, \mu).$$

The latent variables of the model are $\theta = \{\mu, c\}$, including the K class means and the n class assignments. The evidence is defined by:

$$p(x) = \int p(\mu) \prod_{i=1}^n \sum_{c_i} p(c_i) p(x_i | c_i, \mu) d\mu.$$

Analyzing this equation shows that the integrand does not factorize into independent terms for each μ_k , because each μ_k appears in all n terms of the product. Thus, this integral does not reduce to a product of one-dimensional integrals over μ_k , which complicates computation. Numerical evaluation of this K -dimensional integral has time complexity on the order of $O(K^n)$.

By expanding the product over the sum and rearranging terms, the evidence can be written as a sum over all possible cluster-assignment configurations:

$$p(x) = \sum_c p(c) \int p(\mu) \prod_{i=1}^n p(x_i | c_i, \mu) d\mu.$$

Each individual integral is computable thanks to conjugacy between the Gaussian prior on components and Gaussian likelihood. However, the number of possible cluster assignment configurations is K^n , causing a combinatorial explosion that makes evidence computation exponential in K , and therefore intractable in practice.

3.4 Markov Chain Monte Carlo methods

The impossibility of exactly computing $p(y)$ makes exact inference impractical in most real cases. To address this, Markov Chain Monte Carlo (MCMC) methods (Robert et al., 1999) were developed. The general principle is to generate a sequence of random variables $\{\theta^{(n)}\}_{n=1}^N$ forming a Markov chain whose stationary distribution is $\pi(\theta) = p(\theta | y)$. Thus, for any estimator that is a function of θ , the posterior expectation can be approximated by an empirical average:

$$\mathbb{E}_{p(\theta|y)}[f(\theta)] = \int f(\theta) p(\theta | y) d\theta \approx \frac{1}{N} \sum_{n=1}^N f(\theta^{(n)}),$$

where $\theta^{(n)}$ is the n -th sample of the chain. A few methods are presented below as examples.

The Metropolis-Hastings method (Metropolis et al., 1953; Hastings, 1970) is based on defining a proposal distribution $q(\theta' | \theta)$. At each iteration, a new sample θ' is proposed from $q(\theta' | \theta^{(n)})$, then accepted with acceptance probability $\alpha(\theta^{(n)}, \theta')$:

$$\alpha(\theta^{(n)}, \theta') = \min \left\{ 1, \frac{\pi(\theta') q(\theta^{(n)} | \theta')}{\pi(\theta^{(n)}) q(\theta' | \theta^{(n)})} \right\},$$

where $\pi(\theta) = p(\theta | y)$ is the target density (the posterior). If sample θ' is accepted, set $\theta^{(n+1)} = \theta'$; otherwise, keep the old one $\theta^{(n+1)} = \theta^{(n)}$.

The Gibbs method (Gelfand, 2000) is a special case of Metropolis-Hastings where the proposal distribution is chosen as each parameter's full conditional distribution. In its simplest form, each component of vector θ is sequentially updated by sampling from its corresponding conditional law. This removes the need to compute an explicit acceptance ratio, because the proposal is always perfectly aligned with the target density for the updated component. This implementation simplicity makes Gibbs sampling very popular, especially in hierarchical modeling or in cases where conditionals are analytically available.

Among major improvements aimed at accelerating or stabilizing convergence are Hamiltonian Monte Carlo (HMC) (Duane et al., 1987; Girolami and Calderhead, 2011), which uses Hamiltonian dynamics to explore parameter space more efficiently, and stochastic variants (e.g., SGLD, SGHMC) (Chen et al., 2014; Welling and Teh, 2011), adapted to large datasets through mini-batch sampling techniques.

Despite their effectiveness, these methods are costly, and the number of samples required to satisfactorily explore the distribution often grows sharply with problem dimension, a phenomenon known as the curse of dimensionality. In deep neural networks or very high-dimensional problems, obtaining reliable sampling can become prohibitive. In addition, many passes (burn-in, thinning, etc.) are sometimes required to ensure convergence to the stationary distribution, further increasing computational cost. Facing these limitations, variational inference emerged as a faster alternative, especially when parameter dimension is high or when online estimation is required. The idea is to approximate the posterior with a parametric family of distributions $q(\theta)$, chosen to be both close to the true distribution and analytically tractable. This approach bypasses the difficulty of explicit $p(y)$ computation and provides a reasonable posterior approximation in many

contexts, notably in deep learning. We discuss these techniques in more detail in the next section.

4 Variational Inference

4.1 Principle

Variational inference reformulates the inference problem as an optimization problem. It is based on approximating the posterior distribution $p(\theta | y)$ by a distribution q that belongs to a tractable family of distributions $\mathcal{Q}$ (Jordan et al., 1999; Blei et al., 2017; Tran et al., 2021). A notion of distance between distributions is needed to establish an ordering relation quantifying the gap between target distribution p and q . The Kullback-Leibler divergence measures similarity between p and q . Although it is sometimes interpreted as a distance, it does not satisfy the definition: it is not symmetric and does not satisfy the triangle inequality. However, it satisfies positivity, separation, and additivity. Positivity follows from Jensen's inequality.

Thus, our objective is to find the minimum of the Kullback-Leibler (KL) divergence between q and $p(\theta | y)$, defined by:

$$\text{KL}(q||p(\cdot | y)) = \int q(\theta) \log \frac{q(\theta)}{p(\theta | y)} d\theta.$$

The optimal distribution q^* is obtained by solving the following optimization problem:

$$q^* = \arg \min_q \text{KL}(q||p(\cdot | y)).$$

However, KL generally cannot be computed analytically. We therefore need a lower bound on which optimization is feasible. The KL divergence can be rewritten using the distribution definition:

$$\text{KL}(q||p(\cdot | y)) = \int q(\theta) \log \frac{q(\theta)}{p(y, \theta)} d\theta + \log p(y).$$

Rearranging terms gives the following fundamental identity:

$$\log p(y) = \text{KL}(q||p(\cdot | y)) + \underbrace{\mathbb{E}_q[\log p(y, \theta) - \log q(\theta)]}_{\mathcal{L}(q)}. \quad (1)$$

The term $\mathcal{L}(q)$ is called the Evidence Lower Bound (ELBO). Since KL divergence is positive, minimizing KL is equivalent to maximizing this lower bound:

$$q^* = \arg \max_q \mathcal{L}(q).$$

The intuition behind ELBO optimization can be seen by rewriting this quantity as the sum of expected data likelihood and the Kullback-Leibler divergence between prior p and variational distribution q :

$$\mathcal{L}(q) = \mathbb{E}_q[\log p(y, \theta)] - \text{KL}(q||p)$$

Optimizing this objective encourages distribution $q(\theta)$ to concentrate mass on specific values of θ . The first term, an expected likelihood, favors densities assigning high probability to latent-variable configurations that explain observations. The second term, the negative divergence between the variational density and the prior, encourages closeness to the prior. Hence, the variational objective reflects the classical likelihood-prior tradeoff.

4.2 Variational family

A common strategy in variational inference is to restrict the family of admissible distributions $\mathcal{Q}$ to a specific parametric class $\{q_\lambda \mid \lambda \in E\}$, where $E \subset \mathbb{R}^d$ for some $d \in \mathbb{N}^*$. This formulation represents each distribution q_λ in the family by a parameter vector λ , called variational parameters. This approach is known as fixed-form variational approximation.

This approach provides high expressiveness for the variational family, but it is costly. Another common approach is the mean-field approximation. It assumes a variational family that factorizes across latent variables,

$$q_\lambda(\theta) = \prod_{i=1}^d q_i(\theta_i, \lambda_i).$$

This choice significantly reduces expressiveness because it introduces a conditional independence assumption between latent variables. However, it exploits this structure to reduce computational cost. This approach is not developed further here (Blei et al., 2017).

It is possible to assume that the mapping $\lambda \mapsto q_\lambda$ is injective without loss of generality. If distinct parameters $\lambda_1 \neq \lambda_2$ defined the same distribution, we could restrict space E to remove this redundancy and obtain injectivity. Restricting this mapping to its image then makes it bijective.

In this framework, the ELBO function, initially defined on the space of distributions,

$$\mathcal{L} : q \mapsto \mathbb{E}_q [\log p(y, \theta) - \log q(\theta)],$$

can be viewed as a function defined on parameter space:

$$\mathcal{L} : \lambda \mapsto \mathbb{E}_{q_\lambda} [\log p(y, \theta) - \log q_\lambda(\theta)].$$

In the following, unless otherwise stated, $\mathcal{L}(\lambda)$ denotes ELBO as a function of variational parameters λ , which allows optimization to be expressed directly in parameter terms rather than distribution terms.

4.3 Optimization

In complex cases, ELBO optimization often relies on gradient descent. The algorithm pseudo-code is shown below.

Algorithm 1 Variational optimization by gradient descent

Require: Objective function $\mathcal{L}(\lambda)$, learning rate η_t , number of iterations T

- 1: Initialize parameters $\lambda_0 \in E$
 - 2: **for** $t = 0, 1, \dots, T - 1$ **do**
 - 3: Compute gradient: $g_t = \nabla_\lambda \mathcal{L}(\lambda_t)$
 - 4: Update parameters: $\lambda_{t+1} = \lambda_t + \eta_t g_t$
 - 5: **end for**
 - 6: **return** λ_T
-

The classical parameter update is:

$$\lambda_{t+1} = \lambda_t + \eta_t \nabla_{\lambda} \mathcal{L}(\lambda_t),$$

where η_t is the learning rate. This algorithm (Boyd and Vandenberghe, 2004) is designed for problems where the objective function is convex and differentiable. In that context, choosing a sequence η_t through exact line search or backtracking guarantees convergence to the global optimum. These sequences are adaptive step-size strategies that ensure efficient and stable descent of the objective, balancing convergence speed and precision. Convergence order depends on Hessian conditioning near the minimum.

For most problems, the ELBO objective is not convex with respect to parameters. Therefore, there is no guarantee of convergence to the global minimum. In practice, however, this algorithm gives satisfactory results in reasonable time.

Stopping criterion

So far, the stopping condition has not been discussed. The algorithm iterates until step T , but no method to choose T has been provided. The idea is that at convergence, $\mathcal{L}(\lambda_t)$ is approximately unchanged. Thus, the algorithm is stopped if this value no longer improves, i.e., does not increase, after P iterations (Tran et al., 2021). P is called the patience parameter. However, the approximation of this quantity,

$$\widehat{\mathcal{L}(\lambda_t)} = \frac{1}{S} \sum_{i=1}^S \log \frac{p(y, \theta)}{q_{\lambda}(\theta)}, \quad \theta_i \sim q_{\lambda},$$

is too noisy for that. Therefore, the quantity used for the criterion is the moving average of the previous estimator, i.e.,

$$\overline{\mathcal{L}(\lambda_t)} = \frac{1}{t_W} \sum_{k=1}^{t_W} \widehat{\mathcal{L}(\lambda_{t-k+1})},$$

with t_W the number of samples used in the moving average. The algorithm therefore stops after P iterations during which the moving average does not decrease.

Learning rate

Note that exact line search or backtracking methods mentioned above are costly and inefficient in non-convex settings. In practice, progressively decreasing this rate is necessary so parameter updates become smaller and smaller. This enables an exploratory phase that helps escape shallow local minima, followed by an exploitation phase that stabilizes the solution. Various update strategies exist, and the best strategy depends on the problem (Smith, 2017; You et al., 2019; d'Ascoli et al., 2022).

Moreover, not all coordinates of vector λ need the same learning rate. Adaptive learning-rate strategies should therefore be used (Zaheer et al., 2018). Common methods include ADAM (Kingma and Ba, 2014) and AdaGrad (Duchi et al., 2011). These methods normalize the gradient by the variance of the corresponding coordinate, estimated using moving averages.

Natural gradient

It was assumed that a small Euclidean distance between λ and λ' implies a small difference between q_λ and $q_{\lambda'}$. Gradient descent indeed makes "small" steps followed by successive reevaluations to explore space progressively. But instead of descending directly in the distribution space q_λ , we replaced it with descent in parameter space λ , relying on the idea that there is a bijection between λ and q_λ . Thus, we implicitly assumed that the notion of a "small" move is preserved under this change of variable. More precisely, we assumed that parameter space and the corresponding distribution space are locally isomorphic enough. This is not guaranteed: a small Euclidean distance in parameter space can correspond to traveling a large distance in distribution space. The set of distributions $\{q_\lambda\}$ is not Euclidean; it is a Riemannian differentiable manifold (Amari and Nagaoka, 2000), and this geometry should be accounted for to describe proximity between distributions more faithfully.

A Riemannian manifold $(\mathcal{M}, g)$ is a differentiable manifold $\mathcal{M}$ equipped with a metric tensor g defining a local distance between points. In our case, the space of parametric distributions $\{q_\lambda\}$ can be equipped with the Fisher-Rao metric, given by the Fisher information matrix:

$$g_\lambda = \mathbb{E}_{q_\lambda} [\nabla_\lambda \log q_\lambda(\theta) \nabla_\lambda \log q_\lambda(\theta)^\top].$$

This metric encodes local curvature of the distribution space and allows gradient direction adjustment to better respect the underlying geometry. Thus, to account for this geometric structure, the space metric, also noted $I_F = \text{Cov}_{q_\lambda}(\nabla_\lambda \log q_\lambda(\theta))$, is used to adapt the gradient direction:

$$\lambda_{t+1} = \lambda_t - \eta I_F^{-1}(\lambda) \widehat{\nabla_\lambda \mathcal{L}(\lambda_t)}.$$

This approach, called natural gradient, improves convergence by better following variational-distribution geometry. It can be viewed as a second-order optimization method, where the Fisher information matrix acts as a Hessian surrogate (Martens, 2020). In high dimension, inverting the Fisher information matrix is costly. Inversion via an LU decomposition costs $O(N^3)$, with $N = |\lambda| = d$ the matrix dimension (Datta, 2010). Since the matrix is symmetric positive definite, its structure can be exploited with Cholesky decomposition. Although faster and more stable, its inversion still has $O(N^3)$ asymptotic cost. However, the goal is not to compute $I_F^{-1}(\lambda)$, but $I_F^{-1}(\lambda) \widehat{\nabla_\lambda \mathcal{L}(\lambda_t)}$. This can be reformulated as:

$$\text{Find } x \in \mathbb{R}^d \text{ such that } I_F(\lambda) \cdot x = \widehat{\nabla_\lambda \mathcal{L}(\lambda_t)}.$$

This is the solution of a linear system $Ax = b$. But inverting the system for a single value of b is inefficient. An effective method for this type of linear system is conjugate gradient (Nazareth, 2009). It is an iterative method that converges in at most $O(N)$ steps, and in practice the number of iterations is often proportional to $\sqrt{\kappa}$, where $\kappa = \frac{|\lambda_{max}|}{|\lambda_{min}|}$ is matrix conditioning. This leads to $O(N^2)$ complexity. Note that this algorithm only requires matrix-vector products $A \cdot v$, which can avoid fully computing the Fisher matrix and allow implicit storage.

A modern method suited to variational families built with neural networks is K-FAC (Kronecker-Factored Approximate Curvature) (Martens and Grosse, 2015; Eschenhagen et al., 2023). Its principle is to approximate the Fisher information matrix by exploiting neural-network structure. More precisely, K-FAC models the Fisher matrix as block-diagonal, each block corresponding to one network layer. Each block is then approximated as a Kronecker product of two smaller matrices representing covariances of activations and gradients for that layer. This factorization enables more efficient inversion of the approximate Fisher matrix, reducing computational cost associated with natural-gradient methods.

More generally, there is a rich literature on efficient computation of natural-gradient descent (Tran et al., 2021; 2020; Martens and Grosse, 2015; Ong et al., 2018).

Gradient of the expectation

ELBO is an expectation. For most complex parametric families, the analytical form of this expectation is unknown, so its gradient cannot be computed directly. The goal is therefore to exchange the gradient and the (possibly generalized) integral.

Assumptions enabling this exchange rely on Lebesgue differentiation under the integral sign. More precisely, let $(\theta, \lambda) \mapsto f(\theta, \lambda)$; under the following conditions:

- Continuity: $\lambda \mapsto f(\theta, \lambda)$ is continuous for $\theta \in \Theta$.
- Differentiability: $\lambda \mapsto f(\theta, \lambda)$ is differentiable almost everywhere for $\theta \in \Theta$.
- Integrable domination: there exists an integrable function $\theta \mapsto g(\theta)$, independent of λ , such that:

$$|\nabla_\lambda f(\theta, \lambda)| \leq g(\theta), \quad \forall \lambda \in \Lambda.$$

then the gradient operator can be exchanged with the integral:

$$\nabla_\lambda \int_{\Theta} f(\theta, \lambda) d\theta = \int_{\Theta} \nabla_\lambda f(\theta, \lambda) d\theta.$$

For ELBO, continuity and differentiability hold. For integrable domination, verification depends on both target and variational distributions. The domination assumption must therefore be checked case by case. Applying the theorem gives:

$$\begin{aligned} \nabla_\lambda \mathcal{L}(\lambda) &= \nabla_\lambda \int q_\lambda(\theta) \cdot [\log p(y, \theta) - \log q_\lambda(\theta)] d\theta \\ &= \int \nabla_\lambda \{q_\lambda(\theta) \cdot [\log p(y, \theta) - \log q_\lambda(\theta)]\} d\theta \\ &= \int \nabla_\lambda q_\lambda(\theta) \cdot [\log p(y, \theta) - \log q_\lambda(\theta)] - q_\lambda(\theta) \nabla_\lambda \log q_\lambda(\theta) d\theta \\ &= \int \nabla_\lambda q_\lambda(\theta) \cdot [\log p(y, \theta) - \log q_\lambda(\theta)] d\theta - \int q_\lambda(\theta) \nabla_\lambda \log q_\lambda(\theta) d\theta. \end{aligned}$$

Noting that,

$$\forall \theta \in \mathbb{R}, \quad q_\lambda(\theta) \nabla_\lambda \log q_\lambda(\theta) = q_\lambda(\theta) \frac{\nabla_\lambda q_\lambda(\theta)}{q_\lambda(\theta)} = \nabla_\lambda q_\lambda(\theta)$$

We then obtain:

$$\begin{aligned} \nabla_\lambda \mathcal{L}(\lambda) &= \int q_\lambda(\theta) \nabla_\lambda \log q_\lambda(\theta) \cdot [\log p(y, \theta) - \log q_\lambda(\theta)] d\theta - \int \nabla_\lambda q_\lambda(\theta) d\theta \\ &= \int q_\lambda(\theta) \nabla_\lambda \log q_\lambda(\theta) \cdot [\log p(y, \theta) - \log q_\lambda(\theta)] d\theta - \underbrace{\nabla_\lambda \int q_\lambda(\theta) d\theta}_{=1} \\ &= \mathbb{E}_{q_\lambda} \left[\nabla_\lambda \log q_\lambda(\theta) \cdot \log \frac{p(y, \theta)}{q_\lambda(\theta)} \right] \end{aligned}$$

The gradient in this form is known as the "score-function gradient". One can sample $\theta \sim q_\lambda(\theta)$ to build a Monte Carlo estimator of this expectation. Let S be the number of generated samples.

$$\widehat{\nabla_\lambda \mathcal{L}(\lambda)} = \frac{1}{S} \sum_{i=1}^S \nabla_\lambda \log q_\lambda(\theta_i) \cdot \log \frac{p(y, \theta_i)}{q_\lambda(\theta_i)}$$

For the estimator to be of good quality, it should be unbiased and have low variance. Let $X(\theta) = \nabla_\lambda \log q_\lambda(\theta) \cdot \log \frac{p(y, \theta)}{q_\lambda(\theta)}$, such that $\widehat{\nabla_\lambda \mathcal{L}(\lambda)} = \frac{1}{S} \sum_{i=1}^S X(\theta_i)$. Then:

$$\begin{aligned} \mathbb{E}_{q_\lambda} [\widehat{\nabla_\lambda \mathcal{L}(\lambda)}] &= \frac{1}{S} \sum_{i=1}^S \mathbb{E}_{q_\lambda} [X(\theta_i)] \\ &= \frac{1}{S} \cdot S \cdot \mathbb{E}_{q_\lambda} [X(\theta)], \quad \text{since } \theta_i \text{ are i.i.d.} \\ &= \mathbb{E}_{q_\lambda} \left[\nabla_\lambda \log q_\lambda(\theta) \cdot \log \frac{p(y, \theta)}{q_\lambda(\theta)} \right] \end{aligned}$$

The estimator is therefore unbiased. Computing the covariance matrix in the general case does not provide additional insight, so it is omitted.

Stochastic variational inference

So far, all data y have been used to evaluate one gradient sample. These data can be both numerous and high-dimensional, which can prevent practical use of this method. To address this issue, stochastic variational inference uses a data subset $\{x_j, y_j\}_{j \in R}$ with $R \subset [1, n]$. In practice, a subset is sampled at each gradient-descent iteration. The data-index subset at iteration t is denoted R_t . The previous estimator expands to:

$$\widehat{\nabla_\lambda \mathcal{L}(\lambda)} = \frac{1}{S} \sum_{i=1}^S \nabla_\lambda \log q_\lambda(\theta_i) \cdot \{\log p(\theta_i) + \log p(y|\theta_i) - \log q_\lambda(\theta_i)\}$$

Since data are assumed independent, we can write:

$$\log p(y|\theta_i) = \sum_{j=1}^n \log p(x_j, y_j|\theta_i)$$

An estimator of this quantity is:

$$\widehat{\log p(y|\theta_i)} = \frac{n}{|R_t|} \sum_{j \in R_t} \log p(x_j, y_j|\theta_i)$$

Since the data are i.i.d., the subsample is also i.i.d. Using this, we obtain that the estimator is unbiased:

$$\begin{aligned} \mathbb{E}[\widehat{\log p(y|\theta_i)}] &= \frac{n}{|R_t|} \sum_{j \in R_t} \mathbb{E}[\log p(x_j, y_j|\theta_i)] \\ &= \frac{n}{|R_t|} \cdot |R_t| \cdot \mathbb{E}[\log p(x_1, y_1|\theta_i)] \\ &= \mathbb{E}\left[\sum_{j=1}^n \log p(x_j, y_j|\theta_i)\right] \\ &= \log p(y|\theta_i) \end{aligned}$$

Thus, the new estimator is:

$$\widehat{\nabla_{\lambda} \mathcal{L}(\lambda)} = \frac{1}{S} \sum_{i=1}^S \nabla_{\lambda} \log q_{\lambda}(\theta_i) \cdot \left\{ \log p(\theta_i) + \frac{n}{|R_t|} \sum_{j \in R_t} \log p(x_j, y_j|\theta_i) - \log q_{\lambda}(\theta_i) \right\}$$

The algorithm can then scale to large datasets.

Variance reduction

In stochastic optimization, algorithm performance depends on the variance of the gradient estimator. The naive estimator introduced above is:

$$\widehat{\nabla_{\lambda} \mathcal{L}(\lambda)} = \frac{1}{S} \sum_{i=1}^S \nabla_{\lambda} \log q_{\lambda}(\theta_i) \cdot \log \frac{p(y, \theta_i)}{q_{\lambda}(\theta_i)}$$

In practice, this estimator often has high variance. Variance-reduction methods are therefore needed.

Control variate A first approach is the control-variate method. Let $X(\theta) = \nabla_{\lambda} \log q_{\lambda}(\theta) \cdot \log \frac{p(y, \theta)}{q_{\lambda}(\theta)}$, the random variable whose expectation is to be estimated. The idea is to introduce a second random variable Y , with zero expectation under q_{λ} (i.e., $\mathbb{E}_{q_{\lambda}}[Y] = 0$). For a sample $\{\theta_i\}_{i=1}^S$ drawn from q_{λ} , the naive estimator is modified by subtracting a control term:

$$\widehat{\nabla_{\lambda} \mathcal{L}(\lambda)}_{\text{CV}} = \frac{1}{S} \sum_{i=1}^S \left[\nabla_{\lambda} \log q_{\lambda}(\theta_i) \log \frac{p(y, \theta_i)}{q_{\lambda}(\theta_i)} - c \cdot Y(\theta_i) \right].$$

Coefficient c is chosen to minimize estimator variance. This estimator is unbiased.

$$\mathbb{E}_{q_\lambda}[X - c \cdot Y(\theta)] = \mathbb{E}_{q_\lambda}[X] - c \cdot \underbrace{\mathbb{E}_{q_\lambda}[Y]}_{=0} = \mathbb{E}_{q_\lambda}[X]$$

Estimator variance is:

$$\mathbb{V}[X - c \cdot Y] = \mathbb{V}[X] - 2c \cdot \text{Cov}[X, Y] + c^2 \cdot \mathbb{V}[Y]$$

This is a polynomial in c , minimized at $c = \frac{\text{Cov}(X, Y)}{\mathbb{V}(Y)}$, with minimum value:

$$\mathbb{V}[X - c \cdot Y] = \mathbb{V}[X] - 2 \cdot \frac{\text{Cov}(X, Y)^2}{\mathbb{V}[Y]}$$

The goal is therefore to maximize the ratio $\frac{\text{Cov}(X, Y)^2}{\mathbb{V}[Y]}$, i.e., high correlation between X and Y relative to variance of Y . Other Monte Carlo variance-reduction methods (e.g., importance sampling, quasi Monte Carlo, Rao-Blackwellization) can also be useful depending on the case.

Pathwise gradient Another, often more effective approach under certain assumptions is the "Pathwise Gradient." First, this approach requires a reparameterization of the variational distribution from a base distribution that does not depend on variational parameters. Second, the log-ratio between model joint density and variational density must be differentiable with respect to latent parameters. Formally,

- There exists a mapping g_λ , a C^1 -diffeomorphism, such that if ε is a random variable with distribution $\pi(\varepsilon)$ independent of λ , then

$$\theta \sim q_\lambda \iff \theta = g_\lambda(\varepsilon) \text{ with } \varepsilon \sim \pi(\varepsilon).$$

- $\theta \rightarrow \log \frac{p(y, \theta)}{q_\lambda(\theta)} \in D^1$

For example, if θ follows $\mathcal{N}(\mu, \sigma^2)$, the following reparameterization can be used:

$$\theta = \mu + \sigma \cdot \varepsilon, \text{ with } \varepsilon \sim \mathcal{N}(0, 1),$$

where μ and σ are the variational parameters, i.e., $\lambda = (\mu, \sigma)$.

The ELBO is reparameterized as:

$$\mathcal{L}(\lambda) = \mathbb{E}_{\theta \sim q_\lambda} \left[\log \frac{p(y, \theta)}{q_\lambda(\theta)} \right] = \mathbb{E}_{\varepsilon \sim \pi} \left[\log \frac{p(y, g_\lambda(\varepsilon))}{q_\lambda(g_\lambda(\varepsilon))} \right]$$

Since the base law no longer depends on λ , we get:

$$\nabla_\lambda \mathcal{L}(\lambda) = \nabla_\lambda \mathbb{E}_{\varepsilon \sim \pi} \left[\log \frac{p(y, g_\lambda(\varepsilon))}{q_\lambda(g_\lambda(\varepsilon))} \right] = \mathbb{E}_{\varepsilon \sim \pi} \left[\nabla_\lambda \left(\log p(y, g_\lambda(\varepsilon)) - \log q_\lambda(g_\lambda(\varepsilon)) \right) \right]$$

On one hand,

$$\nabla_{\lambda} \log p(y, g_{\lambda}(\epsilon)) = \nabla_{\lambda} g_{\lambda}(\epsilon) \cdot \nabla_{\theta} \log p(y, \theta) \Big|_{\theta=g_{\lambda}(\epsilon)}$$

On the other hand,

$$\nabla_{\lambda} \log q_{\lambda}(g_{\lambda}(\epsilon)) = \nabla_{\lambda} g_{\lambda}(\epsilon) \cdot \nabla_{\theta} \log q_{\lambda}(\theta) \Big|_{\theta=g_{\lambda}(\epsilon)}$$

Finally,

$$\nabla_{\lambda} \mathcal{L}(\lambda) = \mathbb{E}_{\epsilon \sim \pi} \left[\nabla_{\lambda} g_{\lambda}(\epsilon) \cdot (\nabla_{\theta} \log p(y, \theta) - \nabla_{\theta} \log q_{\lambda}(\theta)) \Big|_{\theta=g_{\lambda}(\epsilon)} \right]$$

Thus, the estimator is:

$$\widehat{\nabla_{\lambda} \mathcal{L}(\lambda)}_{PG} = \frac{1}{S} \sum_{i=1}^S \left[\nabla_{\lambda} g_{\lambda}(\epsilon_i) \cdot (\nabla_{\theta} \log p(y, \theta) - \nabla_{\theta} \log q_{\lambda}(\theta)) \Big|_{\theta=g_{\lambda}(\epsilon_i)} \right]$$

In practice, this approach often requires significantly fewer samples. Note that its variance is not always lower; this is mainly an empirical observation for common models. Also, the assumptions restrict the models where this method applies.

4.4 Comparison between MCMC and variational inference

Variational inference and Markov Chain Monte Carlo methods are two common approaches to approximating posterior distributions. Each has advantages and drawbacks, so their use is context-dependent.

MCMC methods

Markov Chain Monte Carlo methods, such as Metropolis-Hastings and Gibbs sampling, are designed to produce asymptotically exact samples from posterior distribution $p(\theta | y)$. They rely on constructing a Markov chain whose stationary distribution matches this target distribution. With enough iterations, MCMC theoretically guarantees exact sampling, making it a reference approach in Bayesian inference. These methods are particularly suited to complex distributions, especially when the posterior is multimodal or has strong parameter correlations.

However, convergence of MCMC algorithms can be slow, especially in high dimensions or when model variables are strongly dependent. Moreover, correlation between successive samples means many iterations are required to obtain reliable estimates. Convergence diagnosis is another challenge: it is often difficult to determine whether the chain has reached its stationary distribution. These limitations make MCMC computationally expensive and sometimes impractical for large datasets.

Variational inference

Variational inference takes a different approach by reformulating Bayesian inference as an optimization problem. Instead of sampling directly from the posterior, it approximates it with a distribution $q(\theta)$ adjusted to minimize Kullback-Leibler divergence $D_{KL}(q||p)$. This reformulation replaces a potentially expensive sampling problem with a more efficient

optimization problem, using modern gradient methods and the reparameterization trick. Once optimization is complete, evaluating $q(\theta)$ is instantaneous, which is a major advantage over MCMC chains that require long simulations to generate independent samples. In addition, variational inference is well suited to large datasets through its stochastic extension (SVI), which optimizes ELBO using mini-batches.

However, this approach has limitations. Unlike MCMC methods, which asymptotically converge to the posterior distribution, variational inference provides only an approximation whose quality depends on the chosen family $\mathcal{Q}$. An overly simple variational distribution may fail to capture complex features such as multimodality or strong correlations. Moreover, ELBO optimization can be challenging because it is often non-convex, which may cause convergence issues. In practice, choosing the variational family is a tradeoff between expressiveness and computational efficiency.

5 Sparsity and Gaussian Processes

Gaussian processes (GPs) are distinguished by their high flexibility and their ability to provide uncertainty estimates on predictions. However, with n training data points, constructing and inverting covariance matrix K_{nn} of size $n \times n$ induces a computational cost of $O(n^3)$. This complexity quickly becomes prohibitive as n grows, limiting GP applicability to moderate-size datasets.

A classical approach to address this limitation is to introduce a restricted set of inducing points, denoted X_m , with $m \ll n$. These points, selected or optimized, define a reduced basis used to approximate the full covariance matrix. Inference is then performed in an m -dimensional space, reducing algorithmic complexity to $O(nm^2)$. This performance gain enables applying GPs to larger datasets.

Introducing inducing points nevertheless raises several issues:

1. *Optimal choice of inducing points:*

Positions of points X_m directly affect the quality of the covariance-matrix approximation.

2. *Tradeoff between sparsity and accuracy:*

Excessively reducing m can significantly degrade the model's ability to capture data structure.

3. *Effects on learning and robustness:*

Introducing additional parameters tied to inducing points can increase overfitting risk without proper regularization mechanisms.

5.1 Variational inference for GPs

Variational inference applied to Gaussian processes aims to formulate approximation of the full model by a sparse model as an optimization problem over inducing points. This approach quantifies approximation error while imposing explicit regularization, unlike earlier projection-based methods (Seeger et al., 2003; Snelson and Ghahramani, 2005). It enables joint learning of a variational distribution over process values at inducing points, kernel hyperparameters, and inducing-point locations.

In this setting, the approach proposed by Titsias (Titsias, 2009) introduces an evidence lower bound (*Evidence Lower Bound*, ELBO) as optimization objective. This ELBO includes a penalty term for overly complex configurations, acting as implicit regularization and limiting overfitting.

5.2 Predictive distribution and variational approximation

Predicting Gaussian process values at new points relies on studying the conditional predictive distribution of unknown latent variables. Let z be process values to predict at test points X_* , and let f be the underlying latent function, assumed continuous and noisy

according to model $y = f(x) + \varepsilon$, with $f = (f(x_1), \dots, f(x_n))^T$ the values of this function at training points X . The predictive distribution can be written as:

$$p(z | y) = \int p(z | f) p(f | y) df,$$

where $p(z | f)$ denotes the process distribution evaluated at new points X_* , conditional on latent values f , while $p(f | y)$ is the exact posterior over latent values given noisy observations.

In exact GP settings, this integral is analytically computable but requires inversion of covariance matrix K_{nn} , inducing prohibitive $O(n^3)$ cost. To address this, one approach introduces a reduced set of inducing points X_m , with $m \ll n$, and associated variables f_m , representing process values at points X_m . Extending the previous expression, the predictive distribution becomes:

$$p(z | y) = \int \int p(z | f, f_m) p(f | f_m, y) p(f_m | y) df df_m.$$

Assuming f_m is a sufficient statistic for describing f allows conditional independence of z and f given f_m , i.e.:

$$p(z | f, f_m) = p(z | f_m).$$

Likewise, since y is a noisy version of f , this assumption gives:

$$p(f | f_m, y) = p(f | f_m).$$

Therefore, the predictive distribution can be rewritten as:

$$\begin{aligned} p(z | y) &= \int \int p(z | f_m) p(f | f_m) p(f_m | y) df df_m \\ &= \int p(z | f_m) p(f_m | y) \underbrace{\int p(f | f_m) df}_{=1} df_m \\ &= \int p(z | f_m) p(f_m | y) df_m. \end{aligned} \tag{2}$$

In the variational framework, the exact posterior $p(f_m | y)$ is approximated by a variational distribution $\phi(f_m)$, chosen from a simplified family $\mathcal{Q}$. This family corresponds to Gaussian distributions with mean μ and covariance matrix Λ . To connect with notation from Section 4, the equivalences are:

$$\begin{aligned} \theta &= f_m \\ \lambda &= X_m \\ q(\theta) &= \phi(f_m) \\ p(\theta | y) &= p(f_m | y). \end{aligned}$$

The objective is to minimize the Kullback-Leibler divergence between $\phi(f_m)$ and $p(f_m | y)$, by optimizing the parameters of ϕ . The dependency between these parameters and points X_m will be made explicit later.

The joint variational distribution and its marginal over z are:

$$\begin{aligned} q(z, f_m) &= p(z | f_m) \phi(f_m), \\ q(z) &= \int q(z, f_m) df_m. \end{aligned}$$

Injecting this approximation into expression (2) yields the approximate predictive distribution:

$$p(z | y) \approx \int p(z | f_m) \phi(f_m) df_m = q(z) \quad (3)$$

Applying standard GP prediction formulas to expression (3), we can compute the mean and covariance functions associated with predictive distribution $q(z)$. The conditional law $z | f_m$ is:

$$z | f_m \sim \mathcal{N}(K_{xm} K_{mm}^{-1} f_m, k(\mathbf{x}, \mathbf{x}) - K_{xm} K_{mm}^{-1} K_{mx}).$$

Predictive mean $m_y^q(\mathbf{x})$ is obtained by:

$$\begin{aligned} m_y^q(\mathbf{x}) &= \mathbb{E}_{\phi(f_m)}[K_{xm} K_{mm}^{-1} f_m] = K_{xm} K_{mm}^{-1} \mu, \\ m_y^q(\mathbf{x}) &= K_{xm} K_{mm}^{-1} \mu. \end{aligned} \quad (4)$$

Predictive covariance decomposes as follows:

$$\begin{aligned} \text{Var}_q[z] &= \mathbb{E}_{\phi(f_m)}[\text{Var}[z | f_m]] + \text{Var}_{\phi(f_m)}[\mathbb{E}[z | f_m]] \\ &= k(\mathbf{x}, \mathbf{x}) - K_{xm} K_{mm}^{-1} K_{mx} + K_{xm} K_{mm}^{-1} \Lambda K_{mm}^{-1} K_{mx}. \end{aligned}$$

The associated covariance function is then:

$$k_y^q(\mathbf{x}, \mathbf{x}') = k(\mathbf{x}, \mathbf{x}') - K_{xm} K_{mm}^{-1} K_{mx'} + K_{xm} K_{mm}^{-1} \Lambda K_{mm}^{-1} K_{mx'}. \quad (5)$$

5.3 Variational lower bound (ELBO)

In this section, the objective is to determine expressions of mean μ and covariance Λ of variational distribution $\phi(f_m)$ that minimize divergence $\text{KL}(\phi(f_m) || p(f_m | y))$. This fully specifies the variational Gaussian process. The Kullback-Leibler divergence is minimized via its lower bound, ELBO, written as:

$$\begin{aligned} \mathcal{L}(\phi(f_m)) &= \mathbb{E}_{\phi(f_m)}[\log p(y, f_m) - \log \phi(f_m)] \\ &= \int \phi(f_m) (\log p(y, f_m) - \log \phi(f_m)) df_m \end{aligned}$$

Expanding $p(y, f_m)$, we get:

$$p(y, f_m) = \int p(y | f) p(f | f_m) p(f_m) df$$

Then, applying the logarithm:

$$\log p(y, f_m) = \log \left(\int p(y | f) p(f | f_m) df \right) + \log p(f_m)$$

This expression is injected into ELBO:

$$\begin{aligned} \mathcal{L}(\phi(f_m)) &= \int \phi(f_m) \left(\log \left(\int p(y | f) p(f | f_m) df \right) + \log p(f_m) - \log \phi(f_m) \right) df_m \\ &= \int \phi(f_m) \left(\log \left(\int p(y | f) p(f | f_m) df \right) + \log \frac{p(f_m)}{\phi(f_m)} \right) df_m \end{aligned}$$

Since the logarithm is concave, Jensen's inequality applies:

$$\mathcal{L}(\phi(f_m)) \geq \int \phi(f_m) \left(\int p(f | f_m) \log p(y | f) df \right) df_m + \int \phi(f_m) \log \frac{p(f_m)}{\phi(f_m)} df_m$$

ELBO is a lower bound of the KL divergence. Since the previous expression is below ELBO, it is also a lower bound. We can therefore rewrite:

$$\mathcal{L}(\phi(f_m)) = \int \phi(f_m) \left(\int p(f | f_m) \log p(y | f) df \right) df_m + \int \phi(f_m) \log \frac{p(f_m)}{\phi(f_m)} df_m$$

Procedure :

The variational bound is now used to recover parameters μ (eq. (4)) et Λ (eq. (5)) of optimal variational distribution $\phi^*(f_m)$. To do so, the following steps are used:

1. Express the integral

$$\int p(f | f_m) \log p(y | f) df$$

in explicit form given by:

$$\log G(f_m, y) = [\log \mathcal{N}(y | \alpha, \sigma^2 I)] - \frac{1}{2\sigma^2} \text{Tr}(K_{nn} - Q_{nn}).$$

with $Q_{nn} = K_{nm} K_{mm}^{-1} K_{mn}$. This expression yields a new formulation of the variational bound:

$$\mathcal{L}(\phi(f_m)) = \int \phi(f_m) \log \left(\frac{G(f_m, y) p(f_m)}{\phi(f_m)} \right) df_m.$$

2. Introduce optimal variational distribution $\phi^*(f_m)$ by minimizing Kullback-Leibler divergence between $\phi(f_m)$ and $\frac{Q(f_m)}{Z}$, where $Q(f_m)$ and Z will be defined later, such that:

$$\mathcal{L}(X_m, \phi) + \frac{1}{2\sigma^2} \text{Tr}(K_{nn} - Q_{nn}) = \log(Z) - KL\left(\phi(f_m) \parallel \frac{Q(f_m)}{Z}\right).$$

By canceling KL divergence, we get:

$$\phi^*(f_m) = \frac{Q(f_m)}{Z}.$$

3. Consequently, an optimal variational lower bound depending on ϕ^* is obtained. A simplified form of this bound is then sought.
4. Finally, it is established that

$$\phi^*(f_m) \propto Q(f_m),$$

which allows identifying parameters μ and Λ .

1. Simplification of integral $\int p(f | f_m) \log p(y | f) df$:

First, $\log p(y | f)$ is computed. Second, expression of $p(f | f_m)$ is derived, then the integral is evaluated.

$$\log p(y | f) = \log \left(\prod_{i=1}^n \mathcal{N}(y_i | f_i, \sigma^2) \right) = \log \left((2\pi\sigma^2)^{-\frac{n}{2}} \exp \left(-\frac{1}{2\sigma^2} (y - f)^\top (y - f) \right) \right).$$

This gives:

$$\log p(y | f) = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} (y - f)^\top (y - f).$$

Conditional distribution $p(f | f_m)$ is a normal distribution with mean and covariance:

$$\alpha = K_{nm} K_{mm}^{-1} f_m = \mathbb{E}_{p(f|f_m)}[f], \quad \Sigma = K_{nn} - Q_{nn} = K_{nn} - K_{nm} K_{mm}^{-1} K_{mn}.$$

Thus:

$$p(f | f_m) = \mathcal{N}(f | \alpha, \Sigma).$$

It is then possible to compute integral $\int p(f | f_m) \log p(y | f) df = \log G(f_m, y)$. Expression $\log G(f_m, y)$ is:

$$\log G(f_m, y) = \int p(f | f_m) \left(-\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} (y - f)^\top (y - f) \right) df.$$

This expression contains: (i) a constant term independent of f , and (ii) a quadratic term in f .

Constant term:

$$\int p(f | f_m) \left(-\frac{n}{2} \log(2\pi\sigma^2) \right) df = -\frac{n}{2} \log(2\pi\sigma^2).$$

Quadratic term:

Expectation of the quadratic term is:

$$\mathbb{E}_{p(f|f_m)}[(y - f)^\top (y - f)] = (y - \alpha)^\top (y - \alpha) + \text{Tr}(K_{nn} - Q_{nn}).$$

Substituting both results:

$$\log G(f_m, y) = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} ((y - \alpha)^\top (y - \alpha) + \text{Tr}(K_{nn} - Q_{nn})).$$

This leads to:

$$\log G(f_m, y) = \log \mathcal{N}(y | \alpha, \sigma^2 I) - \frac{1}{2\sigma^2} \text{Tr}(K_{nn} - Q_{nn}).$$

Thus, the variational bound becomes:

$$\mathcal{L}(\phi(f_m)) = \int \phi(f_m) \log \left(\frac{G(f_m, y) p(f_m)}{\phi(f_m)} \right) df_m.$$

2. Introducing the optimal variational distribution $\phi^*(f_m)$:

The expression of variational bound $\mathcal{L}(\phi(f_m))$ is recalled:

$$\mathcal{L}(\phi(f_m)) = \int \phi(f_m) \left[\log \mathcal{N}(y | \alpha, \sigma^2 I) - \frac{1}{2\sigma^2} \text{Tr}(K_{nn} - Q_{nn}) + \log \frac{p(f_m)}{\phi(f_m)} \right] df_m.$$

This expression can be decomposed as:

$$\mathcal{L}(\phi(f_m)) = \underbrace{\int \phi(f_m) \log \left(\frac{\mathcal{N}(y | \alpha, \sigma^2 I) p(f_m)}{\phi(f_m)} \right) df_m}_S - \frac{1}{2\sigma^2} \text{Tr}(K_{nn} - Q_{nn}).$$

The objective is to maximize this variational bound. Focus is put on the first term, denoted S , which we want to express as a Kullback-Leibler divergence. For this, the following quantities are introduced:

$$Q(f_m) = p(f_m) \mathcal{N}(y | \alpha, \sigma^2 I), \quad S = \int \phi(f_m) \log \left(\frac{Q(f_m)}{\phi(f_m)} \right) df_m.$$

Term S is then related to the Kullback-Leibler divergence between $\phi(f_m)$ and $Q(f_m)/Z$, defined as:

$$KL \left(\phi(f_m) \parallel \frac{Q(f_m)}{Z} \right) = \int \phi(f_m) \log \left(\frac{\phi(f_m)}{Q(f_m)/Z} \right) df_m.$$

Normalization constant Z is:

$$Z = \int p(f_m) \mathcal{N}(y \mid \alpha, \sigma^2 I) df_m.$$

Thus:

$$S = \log(Z) - KL \left(\phi(f_m) \parallel \frac{Q(f_m)}{Z} \right).$$

Optimal case: KL divergence cancellation

Kullback-Leibler divergence is always non-negative and vanishes if and only if:

$$\phi^*(f_m) = \frac{Q(f_m)}{Z} = \frac{p(f_m) \mathcal{N}(y \mid \alpha, \sigma^2 I)}{Z}. \quad (6)$$

Replacing $\phi(f_m)$ by $\phi^*(f_m)$, we obtain:

$$\int \phi^*(f_m) \log \left(\frac{p(f_m) \mathcal{N}(y \mid \alpha, \sigma^2 I)}{\phi^*(f_m)} \right) df_m = \log(Z).$$

Final expression of the optimal variational bound

Thus, for $\phi(f_m) = \phi^*(f_m)$, the variational bound becomes:

$$\mathcal{L}(X_m, \phi^*) = \log(Z) - \frac{1}{2\sigma^2} \text{tr}(K_{nn} - Q_{nn}) = \log \left(\int p(f_m) \mathcal{N}(y \mid \alpha, \sigma^2 I) df_m \right) - \frac{1}{2\sigma^2} \text{tr}(K_{nn} - Q_{nn}).$$

This expression corresponds to variational optimization of the model, where distribution $\phi^*(f_m)$ is chosen to maximize the variational bound.

3. Simplified form of the maximal variational bound :

Observing that Z can be written as $\int p(f_m) \tilde{q}(y \mid f_m) df_m$, with $\tilde{q}$ the conditional density of variable $y = K_{nm} K_{mm}^{-1} f_m + \epsilon$, where $\epsilon \sim \mathcal{N}(0, \sigma^2 I)$ and $f_m \sim \mathcal{N}(0, K_{mm})$, we can establish:

$$Z = \mathcal{N}(y \mid 0, \sigma^2 I + Q_{nn}).$$

This amounts to showing that:

$$y \sim \mathcal{N}(0, \sigma^2 I + Q_{nn}).$$

1. Expectation:

$$\mathbb{E}[y] = K_{nm} K_{mm}^{-1} \mathbb{E}[f_m] = 0.$$

2. Variance:

$$\text{Cov}(y) = K_{nm} K_{mm}^{-1} \text{Cov}(f_m) (K_{nm} K_{mm}^{-1})^\top + \sigma^2 I,$$

with:

$$\text{Cov}(f_m) = K_{mm}.$$

Thus, by symmetry and positivity of K_{nn} , it follows:

$$\text{Cov}(y) = K_{nm} K_{mm}^{-1} K_{mm} (K_{mm}^{-1})^\top K_{nm}^\top + \sigma^2 I = Q_{nn} + \sigma^2 I.$$

Therefore:

$$y \sim \mathcal{N}(0, \sigma^2 I + Q_{nn}), \quad \text{et} \quad Z = \mathcal{N}(y \mid 0, \sigma^2 I + Q_{nn}).$$

The variational bound then becomes:

$$\mathcal{L}(X_m) = \log \mathcal{N}(y \mid 0, \sigma^2 I + Q_{nn}) - \frac{1}{2\sigma^2} \text{tr}(K_{nn} - Q_{nn}).$$

Maximization of this function directly depends on inducing points and their number, allowing optimization of these parameters.

In the variational framework, distribution q is defined as $q(f, f_m) = p(f \mid f_m) \phi^*(f_m)$. Inducing points control flexibility of this variational distribution through conditional mean $K_{nm} K_{mm}^{-1} f_m$, which defines how information in f_m is projected onto all f . The choice of inducing points also affects conditional covariance $K_{nn} - Q_{nn}$, which governs residual uncertainty on f given f_m .

4. Optimized variational distribution $\phi^*(f_m)$:

Points X_m also influence the shape of optimized distribution $\phi^*(f_m)$. From eq. (6), we have:

$$\phi^*(f_m) = \frac{p(f_m) \mathcal{N}(y \mid \alpha, \sigma^2 I)}{Z} \propto p(f_m) \mathcal{N}(y \mid \alpha, \sigma^2 I) = \mathcal{N}(f_m \mid 0, K_{mm}) \mathcal{N}(y \mid \alpha, \sigma^2 I),$$

where:

$$p(f_m) = \mathcal{N}(f_m \mid 0, K_{mm}) = \frac{1}{(2\pi)^{m/2} |K_{mm}|^{1/2}} \exp \left(-\frac{1}{2} f_m^\top K_{mm}^{-1} f_m \right),$$

et :

$$\mathcal{N}(y \mid \alpha, \sigma^2 I) = \frac{1}{(2\pi\sigma^2)^{n/2}} \exp \left(-\frac{1}{2\sigma^2} (y - \alpha)^\top (y - \alpha) \right),$$

with $\alpha = K_{nm} K_{mm}^{-1} f_m$.

Expanding the quadratic term:

$$(y - \alpha)^\top (y - \alpha) = y^\top y - 2y^\top K_{nm} K_{mm}^{-1} f_m + f_m^\top K_{mm}^{-1} K_{mn} K_{nm} K_{mm}^{-1} f_m.$$

This expression allows rewriting $\phi^*(f_m)$ in exponential form:

$$\phi^*(f_m) \propto \exp \left(-\frac{1}{2} f_m^\top \Lambda f_m + f_m^\top b \right),$$

where:

$$\Lambda = K_{mm}^{-1} + \frac{1}{\sigma^2} K_{mm}^{-1} K_{mn} K_{nm} K_{mm}^{-1} \quad (7)$$

et :

$$b = \frac{1}{\sigma^2} K_{mm}^{-1} K_{mn} y.$$

Completing the square in f_m , we get:

$$-\frac{1}{2} f_m^\top \Lambda f_m + f_m^\top b = -\frac{1}{2} (f_m - \mu)^\top \Lambda (f_m - \mu) + \frac{1}{2} \mu^\top \Lambda \mu,$$

with:

$$\mu = \Lambda^{-1} b.$$

Since term $\frac{1}{2} \mu^\top \Lambda \mu$ does not depend on f_m , it can be ignored for the normalized form of the distribution.

Matrix Λ , defined by equation (7), can be rewritten as:

$$\Lambda = K_{mm}^{-1} \left(I + \frac{1}{\sigma^2} K_{mn} K_{nm} \right) K_{mm}^{-1}.$$

En posant :

$$\boxed{\Sigma = I + \frac{1}{\sigma^2} K_{mn} K_{nm}},$$

we obtain:

$$\boxed{\Lambda = K_{mm}^{-1} \Sigma K_{mm}^{-1} \Rightarrow \Lambda^{-1} = K_{mm} \Sigma^{-1} K_{mm}.$$

Vector b is:

$$b = \frac{1}{\sigma^2} K_{mm}^{-1} K_{mn} y,$$

hence:

$$\boxed{\mu = \Lambda^{-1} b = \frac{1}{\sigma^2} K_{mm} \Sigma^{-1} K_{mn} y.}$$

Optimized distribution $\phi^*(f_m)$ then takes the form:

$$\phi^*(f_m) \propto \exp \left(-\frac{1}{2} (f_m - \mu)^\top \Lambda (f_m - \mu) \right),$$

which corresponds to:

$$\phi^*(f_m) = \mathcal{N}(f_m \mid \mu, \Lambda).$$

The variational Gaussian process is thus fully specified.

5.4 Inducing point selection

A commonly used method for selecting inducing points is gradient-based optimization. This approach maximizes variational lower bound $\mathcal{L}(\phi)$ with respect to inducing-point positions X_m . However, this method can be costly due to high problem dimensionality, especially when the number of inducing points m is large.

An alternative is greedy selection Titsias (2009). This method exploits the fact that $\mathcal{L}(\phi)$ is an increasing function when new inducing points are added. The idea is to start with an empty set of inducing points and iteratively add points that maximize the increase in $\mathcal{L}(\phi)$. This approach is faster than gradient optimization, but can lead to suboptimal solutions.

Finally, another method uses a sampling-based approach. For example, inducing points can be selected by random subsampling of training data or by using clustering techniques, such as k -means++ Arthur and Vassilvitskii (2006), to identify the most representative points.

5.5 Stochastic Variational Inference (SVI)

To make variational inference for GPs scalable to larger datasets, Hensman et al. (2013) proposes an extension of the variational method introduced by Titsias (2009). This approach enables Stochastic Variational Inference (SVI), meaning variational parameters are updated from data mini-batches instead of using the full training sample at each iteration.

Although the variational bound from Titsias (2009) provides a good approximation, it is said to be collapsed since latent function f is globally integrated out. The variational bound obtained by Titsias is:

$$\mathcal{L}_{\text{Titsias}} = \underbrace{\log \mathcal{N}(\mathbf{y} \mid \mathbf{0}, Q_{nn} + \sigma_n^2 \mathbf{I})}_{\text{log-likelihood term}} - \underbrace{\frac{1}{2\sigma_n^2} \text{tr}(K_{nn} - Q_{nn})}_{\text{correction term}},$$

This form is called collapsed because it does not allow explicit decomposition of likelihood as a sum over observations. It is not possible to efficiently use mini-batches to update parameters with this global integration, because full likelihood factorization still depends on the whole observation set at each step.

The work of Hensman et al. (2013) is based on reformulating the variational bound to allow factorization compatible with stochastic optimization. We consider a Gaussian process with inducing points X_m and f_m , the function values at inducing points. In the variational setting, a Gaussian distribution is introduced to approximate the posterior, parameterized by mean $\mathbf{m}$ and covariance $\mathbf{S}$:

$$\phi(\mathbf{f}_m) = \mathcal{N}(\mathbf{m}, \mathbf{S}).$$

Applying standard ELBO construction, we obtain:

$$\mathcal{L} = \mathbb{E}_{\phi(\mathbf{f}_m)} [\log p(\mathbf{y} \mid \mathbf{f}_m)] + \log p(\mathbf{f}_m) - \log \phi(\mathbf{f}_m)$$

Assuming the likelihood factorizes over observations $\{(x_i, y_i)\}_{i=1}^n$, this bound can be rewritten as a sum of local terms.

For each point x_i , define the vector of cross-covariances with inducing points:

$$\mathbf{k}_i \equiv \begin{bmatrix} k(x_i, X_{m_1}) \\ \vdots \\ k(x_i, X_{m_m}) \end{bmatrix} \in \mathbb{R}^m.$$

From the GP conditioning property, distribution of $f(x_i)$ conditional on $\mathbf{f}_m$ is:

$$f(x_i) | \mathbf{f}_m \sim \mathcal{N}(\mathbf{k}_i^\top \mathbf{K}_{mm}^{-1} \mathbf{f}_m, k(x_i, x_i) - \mathbf{k}_i^\top \mathbf{K}_{mm}^{-1} \mathbf{k}_i).$$

In Gaussian regression, i.i.d. additive noise is added: $\varepsilon_i \sim \mathcal{N}(0, \sigma_n^2)$, hence observation distribution:

$$y_i = f(x_i) + \varepsilon_i \Rightarrow y_i \sim \mathcal{N}(\mathbf{k}_i^\top \mathbf{K}_{mm}^{-1} \mathbf{f}_m, \sigma_n^2).$$

Expectation with respect to $\phi(\mathbf{f}_m)$ then rewrites the variational bound as:

$$\mathcal{L}_2 = \sum_{i=1}^n \left\{ \log \mathcal{N}(y_i | \mathbf{k}_i^\top \mathbf{K}_{mm}^{-1} \mathbf{m}, \sigma_n^2) - \frac{1}{2} \sigma_n^2 \tilde{k}_{i,i} - \frac{1}{2} \text{tr}(\mathbf{S} \mathbf{\Lambda}_i) \right\} - \text{KL}(\phi(\mathbf{f}_m) \| p(\mathbf{f}_m)), \quad (8)$$

where:

$$\tilde{k}_{i,i} = \mathbf{k}_i^\top \mathbf{K}_{mm}^{-1} \mathbf{k}_i, \quad \mathbf{\Lambda}_i = \sigma_n^2 \mathbf{K}_{mm}^{-1} \mathbf{k}_i \mathbf{k}_i^\top \mathbf{K}_{mm}^{-1}.$$

Kullback-Leibler divergence between two normal distributions can be computed explicitly:

$$\text{KL}(\phi(\mathbf{f}_m) \| p(\mathbf{f}_m)) = \frac{1}{2} \left[\text{tr}(\mathbf{K}_{mm}^{-1} \mathbf{S}) + \mathbf{m}^\top \mathbf{K}_{mm}^{-1} \mathbf{m} - m + \log \frac{|\mathbf{K}_{mm}|}{|\mathbf{S}|} \right].$$

Taking derivatives of $\mathcal{L}_2$ with respect to variational parameters $\mathbf{m}$ and $\mathbf{S}$ gives:

$$\frac{\partial \mathcal{L}_2}{\partial \mathbf{m}} = \sigma_n^{-2} \mathbf{K}_{mm}^{-1} \mathbf{K}_{mn} \mathbf{y} - \mathbf{\Lambda} \mathbf{m}, \quad \frac{\partial \mathcal{L}_2}{\partial \mathbf{S}} = \frac{1}{2} \mathbf{S}^{-1} - \frac{1}{2} \mathbf{\Lambda}.$$

Setting these derivatives to zero leads to optimum:

$$\mathbf{S} = \mathbf{\Lambda}^{-1}, \quad \mathbf{m} = \sigma_n^{-2} \mathbf{\Lambda}^{-1} \mathbf{K}_{mm}^{-1} \mathbf{K}_{mn} \mathbf{y}.$$

In formulation $\mathcal{L}_2$ of equation (8), each sum term can be computed independently, so a subset $\mathcal{B} \subset \{1, \dots, n\}$ of indices can be used. This enables incremental updates of variational parameters using samples (mini-batches). At each iteration, a mini-batch $\mathcal{B}$ is selected, and the following estimator is used:

$$\widehat{\mathcal{L}}_2 = \frac{n}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \left\{ \log \mathcal{N}(y_i | \mathbf{k}_i^\top \mathbf{K}_{mm}^{-1} \mathbf{m}, \sigma_n^2) - \frac{1}{2} \sigma_n^2 \tilde{k}_{i,i} - \frac{1}{2} \text{tr}(\mathbf{S} \mathbf{\Lambda}_i) \right\} - \text{KL}(\phi(\mathbf{f}_m) \| p(\mathbf{f}_m)).$$

This stochastic estimate makes optimization of $\mathcal{L}_2$ scalable and suitable for learning on very large datasets.

6 Deep Kernel Learning

6.1 The GP problem in high dimensions

Gaussian Processes are widely used in machine learning to model complex relationships in data thanks to their flexibility and rigorous probabilistic framework. However, they suffer from a major limitation when applied to high-dimensional data: the difficulty of defining an effective kernel that properly captures the underlying nonlinear data structure.

To address this issue, several research directions propose adapting or extending kernel structure to better represent intrinsic data variability (Duvenaud et al. (2013), Wilson and Adams (2013)).

The method used in this report to address this issue is Deep Kernel Learning Wilson et al. (2016b).

6.2 Principle of Deep Kernel Learning

Deep Kernel Learning (DKL) is a hybrid approach that combines deep neural networks with Gaussian processes to overcome GP limitations. The neural network learns to transform raw data into a richer, more structured representation called latent space. This latent space is designed to capture complex data aspects. Thus, instead of applying a GP directly to high-dimensional raw data, the GP operates on lower-dimensional neural-network representations where nonlinear relationships are better structured.

In Wilson et al. (2016b), inducing points are also introduced to improve GP sparsity. Although latent space has lower dimension than input data, this dimension can still be high enough that GP training remains computationally expensive. Therefore, these inducing points summarize information in latent space, reducing covariance-matrix computation cost. Inducing-point selection builds an approximation that preserves underlying structure while significantly simplifying the problem.

Moreover, Wilson et al. (2016b) uses KISS-GP (Kernel Interpolation for Scalable Structured Gaussian Processes) Wilson and Nickisch (2015) rather than classical sparse-GP approaches. KISS-GP exploits the underlying structure of regular grids to interpolate covariances efficiently, allowing significant reduction in computational complexity. KISS-GP constructs a covariance-matrix approximation from a point grid that facilitates interpolation for new points and makes covariance inversion faster and less expensive.

6.3 Mathematical formulation

Consider a GP with kernel k and hyperparameters $\theta = \{l, \sigma_f, \sigma_n\}$. Also consider data $Y = (y_1, \dots, y_n)^\top$ and $X = (x_1, \dots, x_n)^\top$. Let ψ_w be a neural network with weights w . Kernel inputs are then transformed by the neural network:

$$k(x_i, x_j | \theta) \rightarrow k(h_i | \theta, w),$$

with $h_i = \psi_w(x_i)$. We then denote:

$$K_{nn} = \left(k(h_i | \theta, w) \right)_{1 \leq i, j \leq n}$$

Since computing inverse K_{nn}^{-1} is too costly for large training datasets, this quantity must be approximated.

In the original version of (Wilson et al., 2016b), matrix K_{nn} is approximated using KISS-GP, which provides a direct covariance-matrix approximation. However, according to (Wilson et al., 2016a), this method is not applicable to very high-dimensional data. The method used for SVDKL introduces inducing points f_m and optimizes them. This is the sparse-GP method introduced in the previous section. The same reasoning applies except that $\psi_w(X)$ is substituted for X . We recall the formalism, with p the model, q the variational model, and $\mathcal{L}$ the ELBO.

The joint probabilistic model is:

$$p(y, f, f_m) = p(y | f) p(f | f_m) p(f_m)$$

with:

- Distribution of observations conditional on f :

$$p(y | f) = \prod_{i=1}^n p(y_i | f(x_i))$$

- GP distribution on observations conditioned on f_m :

$$p(f | f_m) = \mathcal{N}(f | K_{nm} K_{mm}^{-1} f_m, K_{nn} - K_{nm} K_{mm}^{-1} K_{mn})$$

- Prior distribution on inducing points f_m :

$$p(f_m) = \mathcal{N}(f_m | 0, K_{mm})$$

The variational model $q(f, f_m)$ is:

$$q(f, f_m) = p(f | f_m) \phi(f_m)$$

with $\phi(f_m)$ a Gaussian variational distribution parameterized by $\lambda = (\mu, \Sigma)$,

$$\phi(f_m) = \mathcal{N}(f_m | \mu, \Sigma)$$

The optimization problem is:

$$q^* = \arg \min_q \text{KL}(q(f, f_m) || p(f, f_m | y))$$

Finally, ELBO is:

$$\mathcal{L} = \log(\mathcal{N}(Y | 0, \sigma_n^2 I + Q_{nn})) - \frac{1}{2\sigma^2} \text{Tr}(K_{nn} - Q_{nn})$$

with:

$$\begin{aligned}Q_{nn} &= K_{nm}K_{mm}^{-1}K_{mn} \\K_{nn} &= k(h, h; \theta) \\K_{nm} &= k(h, X_m; \theta) \\K_{mm} &= k(X_m, X_m; \theta)\end{aligned}$$

6.4 Model optimization

The objective is to optimize ELBO using gradient descent. We need to compute the ELBO gradient with respect to its parameters. Three parameter groups are identified:

θ : GP kernel hyperparameters
 U : inducing points
 w : neural-network weights

Partial derivatives of $\mathcal{L}$ are computed using the chain rule.

6.5 Algorithmic formalization

Three independent algorithmic proposals are given for ELBO optimization.

The first algorithm optimizes all parameters by gradient descent.

Algorithm 2 DKL model optimization with **Sparse GP by gradient descent**

Require: Training set (X, Y) , deep network ψ_w , kernel hyperparameters θ , inducing points X_m , learning rate η , number of iterations T

1: Initialize parameters θ , X_m , and w

2: **for** $t = 1$ **to** T **do**

3: **Forward pass:**

4: Compute deep representation: $h \leftarrow \psi_w(X)$

5: Compute covariance matrices:

$$K_{nn} \leftarrow k(h, h; \theta), \quad K_{nm} \leftarrow k(h, X_m; \theta), \quad K_{mm} \leftarrow k(X_m, X_m; \theta)$$

6: **ELBO computation:**

$$Q_{nn} \leftarrow K_{nm} K_{mm}^{-1} K_{mn}$$

$$\mathcal{L} \leftarrow \log(\mathcal{N}(Y | 0, \sigma_n^2 I + Q_{nn})) - \frac{1}{2\sigma^2} \text{Tr}(K_{nn} - Q_{nn})$$

7: **Gradient computation:**

$$g_\theta \leftarrow \nabla_\theta \mathcal{L}, \quad g_{X_m} \leftarrow \nabla_{X_m} \mathcal{L}, \quad g_w \leftarrow \nabla_w \mathcal{L}$$

8: **Parameter update:**

$$\theta \leftarrow \theta + \eta g_\theta, \quad X_m \leftarrow X_m + \eta g_{X_m}, \quad w \leftarrow w + \eta g_w$$

9: **end for**

10: **return** θ, X_m, w

The second algorithm chooses inducing points at each iteration using K-means, then performs one gradient step.

Algorithm 3 DKL model optimization with **Sparse GP** by gradient descent and **K-means inducing-point selection**

Require: Training set (X, Y) , deep network ψ_w , kernel hyperparameters θ , inducing points X_m , learning rate η , number of iterations T

1: Initialize parameters θ , X_m , and w

2: **for** $t = 1$ **to** T **do**

3: **Forward pass:**

4: Compute deep representation: $h \leftarrow \psi_w(X)$

5: Update inducing points with k-means: $X_m \leftarrow \text{KMeans}(h, m)$

6: Compute covariance matrices:

$$K_{nn} \leftarrow k(h, h; \theta), \quad K_{nm} \leftarrow k(h, X_m; \theta), \quad K_{mm} \leftarrow k(X_m, X_m; \theta)$$

7: **ELBO computation:**

$$Q_{nn} \leftarrow K_{nm} K_{mm}^{-1} K_{mn}$$

$$\mathcal{L} \leftarrow \log(\mathcal{N}(Y | 0, \sigma_n^2 I + Q_{nn})) - \frac{1}{2\sigma^2} \text{Tr}(K_{nn} - Q_{nn})$$

8: **Gradient computation:**

$$g_\theta \leftarrow \nabla_\theta \mathcal{L}, \quad g_w \leftarrow \nabla_w \mathcal{L}$$

9: **Parameter update:**

$$\theta \leftarrow \theta + \eta g_\theta, \quad w \leftarrow w + \eta g_w$$

10: **end for**

11: **return** θ , X_m , w

The third algorithm chooses inducing points at each iteration using greedy selection, then performs one gradient step.

Algorithm 4 DKL model optimization with **Sparse GP by gradient descent and greedy inducing-point selection**

Require: Training set (X, Y) , deep network ψ_w , kernel hyperparameters θ , inducing points X_m , learning rate η , number of iterations T

```

1: Initialize parameters  $\theta$ ,  $X_m$ , and  $w$ 
2: for  $t = 1$  to  $T$  do
3:   Forward pass:
4:     Compute deep representation:  $h \leftarrow \psi_w(X)$ 
5:     Greedy inducing-point selection:
6:        $X_m \leftarrow \emptyset$ 
7:       for  $i = 1$  to  $m$  do
8:          $bestELBO \leftarrow -\infty$ 
9:          $bestCandidate \leftarrow \text{None}$ 
10:        for each  $x$  in  $h \setminus X_m$  do
11:           $Z_{candidate} \leftarrow X_m \cup \{x\}$ 
12:          Compute candidate ELBO  $\mathcal{L}_{candidate}$  using  $Z_{candidate}$ 
13:          if  $\mathcal{L}_{candidate} > bestELBO$  then
14:             $bestELBO \leftarrow \mathcal{L}_{candidate}$ 
15:             $bestCandidate \leftarrow x$ 
16:          end if
17:        end for
18:         $X_m \leftarrow X_m \cup \{bestCandidate\}$ 
19:      end for
20:      Compute covariance matrices:

```

$$K_{nn} \leftarrow k(h, h; \theta), \quad K_{nm} \leftarrow k(h, X_m; \theta), \quad K_{mm} \leftarrow k(X_m, X_m; \theta)$$

```

21:   ELBO computation:

```

$$Q_{nn} \leftarrow K_{nm} K_{mm}^{-1} K_{mn}$$

$$\mathcal{L} \leftarrow \log(\mathcal{N}(Y | 0, \sigma_n^2 I + Q_{nn})) - \frac{1}{2\sigma^2} \text{Tr}(K_{nn} - Q_{nn})$$

```

22:   Gradient computation:

```

$$g_\theta \leftarrow \nabla_\theta \mathcal{L}, \quad g_w \leftarrow \nabla_w \mathcal{L}$$

```

23:   Parameter update:

```

$$\theta \leftarrow \theta + \eta g_\theta, \quad w \leftarrow w + \eta g_w$$

```

24: end for

```

```

25: return  $\theta$ ,  $X_m$ ,  $w$ 

```

For all algorithms, parameter updates are kept in their simplest form for clarity. However, since parameters have very different natures, an adaptive learning strategy is required. Natural gradient is not used here for implementation simplicity. Instead, simpler optimizers discussed earlier, such as ADAM or AdaGrad, can be used.

6.6 Toward a scalable extension of DKL: SVDKL

As with sparse Gaussian-process methods, using neural networks in DKL can quickly become expensive as dataset size grows. Kernel computation and joint optimization of network and covariance function can become prohibitive at scale.

To address this issue, stochastic inference techniques can be used to reduce computational cost while preserving predictive performance. In DKL, this is notably realized through *Stochastic Variational Deep Kernel Learning* (SVDKL) introduced in Wilson et al. (2016a). The idea is to adjust problem formulation, similarly to sparse Gaussian-process methods (see Section 5), to obtain an approximate version of model variables. This enables mini-batch training and easier handling of large data volumes. In this way, the representational power of deep networks is combined with the scalability of stochastic variational inference.

In SVDKL, a sparse Gaussian-process layer is integrated *after* the deep neural network, following a scheme similar to that described in Section 5. Inducing points provide an approximation of the posterior distribution using fewer resources than a full Gaussian process. This approximation relies on a variational-inference procedure that jointly optimizes the Gaussian posterior approximation and network parameters. In addition, mini-batch implementation reduces computation time since each gradient-descent iteration processes only a fraction of the data.

7 Experiments and Results

7.1 Dataset: Olivetti Faces

The *Olivetti Faces* dataset, provided by the `sklearn.datasets` library, contains 400 grayscale images with resolution 64×64 pixels, representing 40 different individuals. Each subject is photographed under varying facial expressions and lighting conditions, introducing significant visual diversity.

To design a non-trivial regression task, a data-augmentation strategy was implemented. It consists of applying a random rotation in the interval $[-45^\circ, 45^\circ]$ to each image, then defining the exact rotation angle as the target variable. This task is methodologically interesting because it requires predicting a continuous variable from redundant and structurally similar visual inputs.

The augmented dataset contains 2000 samples. A random split into two distinct subsets was performed, allocating 80 % of data to training and 20 % to performance evaluation.

7.2 Experimental protocol

Evaluation covers the following six models:

- **Gaussian Process (GP)**
- **Neural Network (NN)**
- **Sparse Gaussian Process (SGP)**
- **Stochastic Variational Gaussian Process (SVGP)**
- **Deep Kernel Learning (DKL)**
- **Stochastic Variational Deep Kernel Learning (SVDKL)**

Performance of these approaches is analyzed through an experimental protocol. The following sections present the main methodological elements used to ensure reliable and reproducible evaluation.

First, the architecture of the neural networks used is described in detail, emphasizing their role in extracting intermediate representations from input data. These representations are then exploited by kernel models in the Deep Kernel Learning framework.

Optimization choices are then discussed. L-BFGS and Adam were selected for their convergence and stability performance. Specific precautions taken to stabilize training, especially with complex probabilistic models, are also detailed.

Finally, systematic use of automatic differentiation is highlighted. This tool plays a central role in efficient gradient computation.

7.3 Neural networks

Artificial neural networks (ANNs) form a class of highly flexible parametric models, able to approximate complex functions that are often difficult to represent analytically. Their theoretical interest notably relies on the universal approximation theorem (Cybenko, 1989; Hornik, 1991), according to which a network with one hidden layer and a nonlinear activation function can approximate any continuous function on a compact set with arbitrary precision, provided the hidden layer has enough neurons.

Structure of a fully connected neural network

A one-hidden-layer neural network can be formally modeled as the composition of a linear transformation followed by a nonlinearity:

$$f_{\theta}(x) = W_2 \phi(W_1 x + b_1) + b_2$$

where:

- $x \in \mathbb{R}^{d_{in}}$ denotes model input;
- $W_1 \in \mathbb{R}^{d_h \times d_{in}}$ and $b_1 \in \mathbb{R}^{d_h}$ are hidden-layer parameters;
- $W_2 \in \mathbb{R}^{d_{out} \times d_h}$ and $b_2 \in \mathbb{R}^{d_{out}}$ are output-layer parameters;
- $\phi(\cdot)$ is a nonlinear activation function, typically ReLU defined as $\phi(x) = \max(0, x)$.

The presence of nonlinearity is essential because it gives the network the ability to learn rich and non-trivial representations beyond the limits of purely linear models.

Convolutional neural networks (CNNs)

When data exhibit regular spatial structure, as with images, it is advantageous to exploit this structure using specialized architectures. Convolutional neural networks (CNNs) were designed to take advantage of local invariances and recurring patterns in visual signals LeCun et al. (1998).

A convolutional layer performs a discrete convolution operation between an input image X and a filter W , defined by:

$$(X * W)(i, j) = \sum_{m=-k}^k \sum_{n=-k}^k X(i + m, j + n) W(m, n)$$

This operation extracts local features such as edges, textures, or patterns while limiting parameter count through weight sharing.

Modern CNN architectures are generally organized as successive blocks composed of convolutional layers, nonlinear activation functions, downsampling (*pooling*) operations, then one or more fully connected output modules.

The architecture used in this work relies on three convolutional blocks, defined as follows:

- Each block applies a 2D convolution followed by ReLU activation and max-pooling, progressively reducing spatial resolution;
- A final fully connected layer projects feature maps into a low-dimensional latent space, typically dimension 16.

This architecture aims to extract a compact and informative image representation, facilitating downstream learning with kernel models.

Learning latent representations

To obtain a relevant latent representation of input images, a supervised pretraining phase is performed. The considered architecture consists of a convolutional encoder followed by a dense linear prediction layer. The encoder projects each image $x \in \mathbb{R}^{64 \times 64}$ into a 16-dimensional latent space, denoted $f_\theta(x) \in \mathbb{R}^{16}$, while the output layer predicts the associated rotation angle as a scalar.

This network is trained in supervised mode using a quadratic loss function:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N (y_i - w^\top f_\theta(x_i) - b)^2$$

where $w \in \mathbb{R}^{16}$ and $b \in \mathbb{R}$ are output linear-layer parameters. Once this training phase is complete, the prediction layer is removed and only the convolutional encoder is kept. It is then used to provide latent inputs to hybrid models such as DKL and SVDKL.

This procedure provides informative and stable encoder initialization, avoiding representation collapse (*representation collapse*), where latent vectors converge to a single point. Such collapse leads to a degenerate covariance matrix in kernel models, making training unstable. The pretraining strategy is therefore essential to ensure high-quality initial representations and good convergence of deep probabilistic models.

7.4 Optimization

Optimization of the considered models relies on techniques adapted to the nature and structure of each approach. Two main method families are used: quasi-Newton methods for exact Gaussian-process models, and adaptive stochastic gradient methods for variational and hybrid models.

Quasi-Newton method: L-BFGS

For the exact Gaussian process (GP), model training consists of maximizing marginal likelihood, defined by:

$$\mathcal{L}(\theta) = \log p(y|X, \theta) = -\frac{1}{2} y^\top K_\theta^{-1} y - \frac{1}{2} \log |K_\theta| - \frac{n}{2} \log(2\pi),$$

where θ denotes kernel hyperparameters, including characteristic length scale, signal variance, and noise variance. This moderate-dimensional optimization problem is solved

using L-BFGS (*Limited-memory Broyden-Fletcher-Goldfarb-Shanno*) (Wright, 2006), which approximates Hessian matrix H_k^{-1} from previous iterations according to:

$$\theta_{k+1} = \theta_k - \eta_k H_k^{-1} \nabla_{\theta} \mathcal{L}(\theta_k)$$

with:

$$s_k = \theta_{k+1} - \theta_k, \quad y_k = \nabla \mathcal{L}(\theta_{k+1}) - \nabla \mathcal{L}(\theta_k), \quad \rho_k = \frac{1}{y_k^{\top} s_k}, \quad V_k = I - \rho_k y_k s_k^{\top}$$

and:

$$H_{k+1}^{-1} = V_k^{\top} H_k^{-1} V_k + \rho_k s_k s_k^{\top}$$

This method ensures fast convergence to a local optimum while maintaining good numerical stability, provided the number of hyperparameters remains reasonable.

Adaptive gradient-descent method: Adam

For variational models (SGP, SVGP) and hybrid models (DKL, SVDKL), parameter space is high-dimensional due to simultaneous presence of neural-network weights, inducing points, and variational parameters. In this setting, stochastic mini-batch optimization is required. Adam (*Adaptive Moment Estimation* (Kingma and Ba, 2014)) is used for its robustness and ability to adapt learning rates dynamically. Gradient moments are estimated as:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla_{\theta} \mathcal{L}(\theta_t) \quad (9)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla_{\theta} \mathcal{L}(\theta_t))^2 \quad (10)$$

These quantities are then bias-corrected:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (11)$$

Parameter updates are then performed via:

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$$

Hyperparameters used in this project are $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\varepsilon = 10^{-8}$.

Pretraining and stabilization for DKL and SVDKL

Joint training of deep hybrid models such as DKL and SVDKL presents specific difficulties, notably due to complex interactions between the neural network and the Gaussian process. To stabilize optimization, a pretraining phase is first carried out: the convolutional neural network is initially trained in supervised mode on the regression task using a simple linear output layer. This step aims to produce an informative and non-degenerate

latent representation.

Once this phase is completed, the linear output layer is removed and the full model (convolutional network and Gaussian process) is jointly trained. Two distinct learning rates are used to improve optimization stability: a lower rate for neural-network weights $\eta_{\text{NN}} = 10^{-3}$ and a higher rate for Gaussian-process hyperparameters $\eta_{\text{GP}} = 10^{-2}$. This differentiation allows:

- preserving stability of representations extracted by the convolutional network;
- accelerating convergence of Gaussian components, thereby improving predictive accuracy.

7.5 Automatic differentiation

Optimization of the models considered in this study relies on efficient evaluation of gradients of objective functions, such as marginal likelihood or ELBO, with respect to model parameters. In this setting, *automatic differentiation* (AD) is a central approach. It enables numerical computation of exact derivatives of complex functions represented as compositions of elementary operations.

General principle

Automatic differentiation is based on the chain rule applied to composed functions. Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be decomposed as a composition of elementary functions:

$$f = f_K \circ f_{K-1} \circ \dots \circ f_1$$

where each intermediate function $f_k : \mathbb{R}^{d_k} \rightarrow \mathbb{R}^{d_{k+1}}$ is differentiable. The derivative of f with respect to input x is given by the successive product of Jacobian matrices of intermediate functions:

$$\nabla_x f(x) = J_{f_K}(f_{K-1}(\dots f_1(x))) \cdot J_{f_{K-1}}(f_{K-2}(\dots f_1(x))) \dots J_{f_1}(x)$$

where each $J_{f_k}(x)$ denotes Jacobian matrix of f_k at x , i.e.:

$$J_{f_k}(x) = \frac{\partial f_k(x)}{\partial x} \in \mathbb{R}^{d_{k+1} \times d_k}$$

Automatic differentiation modes

Two main modes of automatic differentiation are used in practice, depending on derivative propagation direction.

Forward mode (*forward mode*) Forward mode computes derivatives by propagating input perturbations to output in the natural order of operations:

$$\frac{\partial y}{\partial x} = J_{f_K}(f_{K-1}(\dots f_1(x))) \dots J_{f_2}(f_1(x)) J_{f_1}(x)$$

This mode is particularly suited when the number of inputs is small.

Reverse mode (*reverse mode*) Reverse mode, or backpropagation (*backpropagation*), propagates derivatives in reverse order from output to inputs:

$$\frac{\partial y}{\partial x} = J_{f_1}(x)^\top J_{f_2}(f_1(x))^\top \dots J_{f_K}(f_{K-1}(\dots f_1(x)))^\top$$

The cost of this method depends on the number of outputs (often 1 for a scalar loss), not the number of inputs, making it particularly suitable for optimizing models with many parameters.

Dynamic computational graph in PyTorch

In this project, the `PyTorch` library is used to perform reverse-mode automatic differentiation using a dynamic computational graph. During evaluation of function $f_\theta(x)$, a directed acyclic graph is implicitly built, in which:

- nodes correspond to intermediate variables from elementary operations (products, sums, nonlinear functions);
- edges represent dependencies between these operations.

For example, for function:

$$f(\theta, x) = \theta_1 \cdot x^2 + \theta_2 \cdot \sin(x)$$

the underlying computational graph decomposes into intermediate steps:

$$\begin{aligned} v_1 = x^2 & \rightarrow \frac{\partial v_1}{\partial x} = 2x \\ v_2 = \sin(x) & \rightarrow \frac{\partial v_2}{\partial x} = \cos(x) \\ f = \theta_1 \cdot v_1 + \theta_2 \cdot v_2 & \rightarrow \frac{\partial f}{\partial v_1} = \theta_1, \quad \frac{\partial f}{\partial v_2} = \theta_2 \end{aligned}$$

Backpropagation then computes the total derivative:

$$\frac{\partial f}{\partial x} = \theta_1 \cdot 2x + \theta_2 \cdot \cos(x)$$

This process is fully automated by `PyTorch`. During the forward pass, local derivatives are recorded. During the backward pass, these derivatives are used to efficiently propagate gradients from the loss function to model parameters.

Application to model optimization

Automatic differentiation enables joint optimization of neural-network and Gaussian components by ensuring accurate and differentiable gradient computation. This approach allows use of optimizers such as Adam in a unified framework, while ensuring stability and convergence speed needed to train highly expressive models such as DKL and SVDKL.

7.6 Results

Model performance is evaluated with two criteria: root mean squared error (RMSE) on the test set, and predictive-uncertainty quality measured via marginal variance provided by kernel models. Numerical results are shown in Table 1.

Model	RMSE
GP	3.476
NN	2.570
SGP	4.775
SVGP	5.886
DKL	1.687
SVDKL	1.783

Table 1: Root mean squared error of models evaluated on the test set.

The table indicates that hybrid approaches combining a deep neural network with a Gaussian process (DKL, SVDKL) outperform other models in predictive accuracy. DKL achieves the best performance with RMSE of 1.687, closely followed by SVDKL. In contrast, variational models without a pretrained neural encoder (SGP, SVGP) show notable performance degradation.

Figure 1 illustrates predictions on the test set, with 95 % confidence intervals associated with each prediction. These visualizations highlight marked differences in uncertainty calibration across models.

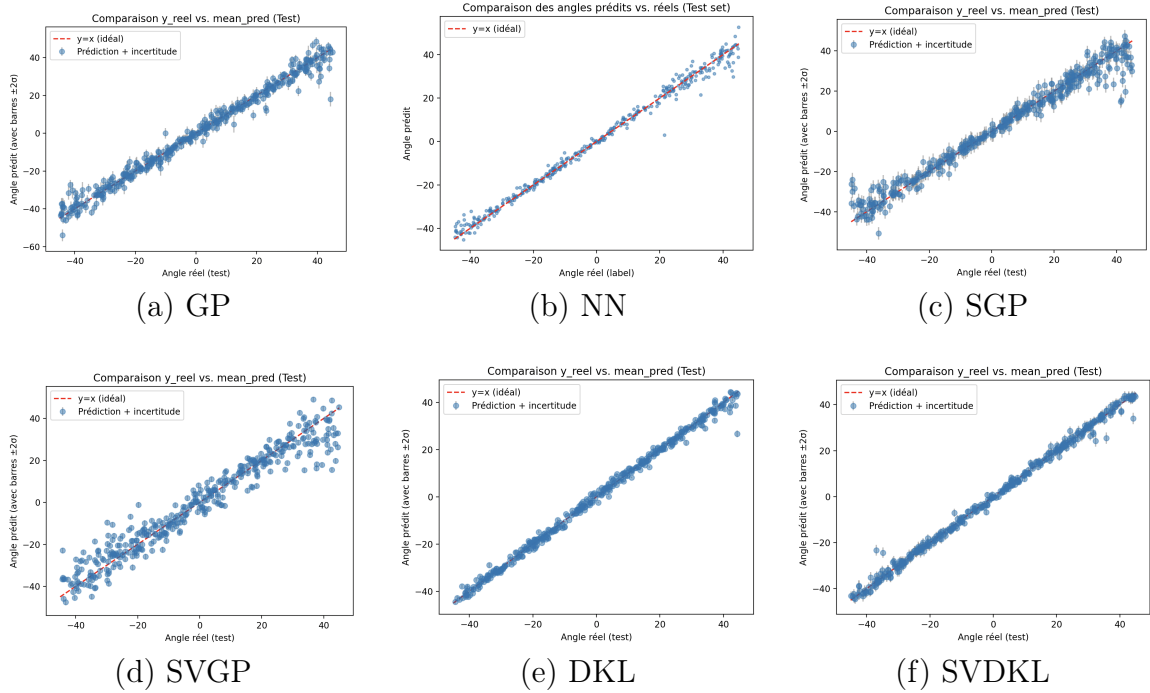


Figure 1: Visual comparison of predictions and 95 % confidence intervals for the evaluated models.

Although DKL and SVDKL provide accurate predictions, displayed confidence intervals seem to underestimate true uncertainty. This phenomenon, already highlighted in the

literature (Foong et al., 2019; Wilson et al., 2016b), is explained by DKL structure itself: the Gaussian process operates in a learned latent space that is often overly contracted, leading to overconfident predictions, especially far from training points.

Predictive quality alone is therefore insufficient to guarantee good calibration, and complementary strategies, such as latent-space regularization or post-hoc calibration, may be considered to correct this bias. More extensive evaluation on synthetic tasks with controlled uncertainty is also needed to confirm these observations.

8 Conclusion

This report provides an in-depth analysis of the effectiveness of hybrid approaches combining Gaussian processes and neural networks to handle complex and large-scale datasets. The analysis of kriging, parametric Bayesian methods, and variational inference made it possible to identify both the strengths of classical approaches and their limitations, especially in terms of computational complexity and uncertainty handling. By combining approximation techniques, such as inducing points, with advanced optimization strategies, this study shows that computational cost can be significantly reduced while maintaining high-quality uncertainty quantification.

The shift to Deep Kernel Learning methods represents a significant advance, offering increased ability to model complex nonlinear relationships through transformation of data into a suitable latent space. This approach makes it possible to fully leverage the strengths of neural networks for learning relevant representations, while integrating the robustness of Gaussian processes for uncertainty estimation.

Furthermore, the detailed examination of variational inference methods highlights the need for a flexible approach to posterior optimization. The use of techniques such as gradient descent adapted to the geometry of distribution spaces opens promising perspectives for high-dimensional problems, while providing practical solutions to bypass the intractability of exact evidence computation.

Overall, the results presented in this work open the way to new research directions, particularly for developing even more scalable models suited to current machine learning challenges.

References

- Shun-ichi Amari and Hiroshi Nagaoka. *Methods of information geometry*, volume 191. American Mathematical Soc., 2000.
- David Arthur and Sergei Vassilvitskii. k-means++: The advantages of careful seeding. Technical report, Stanford, 2006.
- Isabel Beichl and Francis Sullivan. The metropolis algorithm. *Computing in Science & Engineering*, 2(1):65–69, 2000.
- David M Blei, Alp Kucukelbir, and Jon D McAuliffe. Variational inference: A review for statisticians. *Journal of the American statistical Association*, 112(518):859–877, 2017.
- George EP Box and George C Tiao. *Bayesian inference in statistical analysis*. John Wiley & Sons, 2011.
- Stephen P Boyd and Lieven Vandenberghe. *Convex optimization*. Cambridge university press, 2004.
- Tianqi Chen, Emily Fox, and Carlos Guestrin. Stochastic gradient hamiltonian monte carlo. In *International conference on machine learning*, pages 1683–1691. PMLR, 2014.
- Siddhartha Chib and Edward Greenberg. Understanding the metropolis-hastings algorithm. *The american statistician*, 49(4):327–335, 1995.
- George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of control, signals and systems*, 2(4):303–314, 1989.
- Stéphane d’Ascoli, Maria Refinetti, and Giulio Biroli. Optimal learning rate schedules in high-dimensional non-convex optimization problems. *arXiv preprint arXiv:2202.04509*, 2022.
- Biswa Nath Datta. *Numerical linear algebra and applications*. SIAM, 2010.
- Simon Duane, Anthony D Kennedy, Brian J Pendleton, and Duncan Roweth. Hybrid monte carlo. *Physics letters B*, 195(2):216–222, 1987.
- John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of machine learning research*, 12(7), 2011.
- David Duvenaud, James Lloyd, Roger Grosse, Joshua Tenenbaum, and Ghahramani Zoubin. Structure discovery in nonparametric regression through compositional kernel search. In *International Conference on Machine Learning*, pages 1166–1174. PMLR, 2013.
- Runa Eschenhagen, Alexander Immer, Richard Turner, Frank Schneider, and Philipp Hennig. Kronecker-factored approximate curvature for modern neural network architectures. *Advances in Neural Information Processing Systems*, 36:33624–33655, 2023.
- Andrew YK Foong, Yingzhen Li, José Miguel Hernández-Lobato, and Richard E Turner. ‘in-between’ uncertainty in bayesian neural networks. *arXiv preprint arXiv:1906.11537*, 2019.

- Alan E Gelfand. Gibbs sampling. *Journal of the American statistical Association*, 95 (452):1300–1304, 2000.
- Mark Girolami and Ben Calderhead. Riemann manifold langevin and hamiltonian monte carlo methods. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 73(2):123–214, 2011.
- W Keith Hastings. Monte carlo sampling methods using markov chains and their applications. *Biometrika*, 1970.
- James Hensman, Nicolo Fusi, and Neil D Lawrence. Gaussian processes for big data. *arXiv preprint arXiv:1309.6835*, 2013.
- Kurt Hornik. Approximation capabilities of multilayer feedforward networks. *Neural networks*, 4(2):251–257, 1991.
- Michael I Jordan, Zoubin Ghahramani, Tommi S Jaakkola, and Lawrence K Saul. An introduction to variational methods for graphical models. *Machine learning*, 37:183–233, 1999.
- Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- Pierre Simon Laplace. *Théorie analytique des probabilités*. Courcier, 1820.
- Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- James Martens. New insights and perspectives on the natural gradient method. *Journal of Machine Learning Research*, 21(146):1–76, 2020.
- James Martens and Roger Grosse. Optimizing neural networks with kronecker-factored approximate curvature. In *International conference on machine learning*, pages 2408–2417. PMLR, 2015.
- Georges Matheron. *Le krigeage universel*, volume 1. École nationale supérieure des mines de Paris Paris, 1969.
- Nicholas Metropolis, Arianna W Rosenbluth, Marshall N Rosenbluth, Augusta H Teller, and Edward Teller. Equation of state calculations by fast computing machines. *The journal of chemical physics*, 21(6):1087–1092, 1953.
- John L Nazareth. Conjugate gradient method. *Wiley Interdisciplinary Reviews: Computational Statistics*, 1(3):348–353, 2009.
- Victor M-H Ong, David J Nott, and Michael S Smith. Gaussian variational approximation with a factor covariance structure. *Journal of Computational and Graphical Statistics*, 27(3):465–478, 2018.
- Christian P Robert, George Casella, and George Casella. *Monte Carlo statistical methods*, volume 2. Springer, 1999.

- Matthias W Seeger, Christopher KI Williams, and Neil D Lawrence. Fast forward selection to speed up sparse gaussian process regression. In *International Workshop on Artificial Intelligence and Statistics*, pages 254–261. PMLR, 2003.
- Leslie N Smith. Cyclical learning rates for training neural networks. In *2017 IEEE winter conference on applications of computer vision (WACV)*, pages 464–472. IEEE, 2017.
- Edward Snelson and Zoubin Ghahramani. Sparse gaussian processes using pseudo-inputs. *Advances in neural information processing systems*, 18, 2005.
- Michalis K Titsias. Variational model selection for sparse gaussian process regression. *Report, University of Manchester, UK*, 2009.
- M-N Tran, Nghia Nguyen, David Nott, and Robert Kohn. Bayesian deep net glm and glmm. *Journal of Computational and Graphical Statistics*, 29(1):97–113, 2020.
- Minh-Ngoc Tran, Trong-Nghia Nguyen, and Viet-Hung Dao. A practical tutorial on variational bayes. *arXiv preprint arXiv:2103.01327*, 2021.
- Max Welling and Yee W Teh. Bayesian learning via stochastic gradient langevin dynamics. In *Proceedings of the 28th international conference on machine learning (ICML-11)*, pages 681–688. Citeseer, 2011.
- C. Rasmussen & C. Williams. Gaussian processes for machine learning. *The MIT Press*, 2006.
- Andrew Wilson and Ryan Adams. Gaussian process kernels for pattern discovery and extrapolation. In *International conference on machine learning*, pages 1067–1075. PMLR, 2013.
- Andrew Wilson and Hannes Nickisch. Kernel interpolation for scalable structured gaussian processes (kiss-gp). In *International conference on machine learning*, pages 1775–1784. PMLR, 2015.
- Andrew G Wilson, Zhiting Hu, Russ R Salakhutdinov, and Eric P Xing. Stochastic variational deep kernel learning. *Advances in neural information processing systems*, 29, 2016a.
- Andrew Gordon Wilson, Christoph Dann, and Hannes Nickisch. Thoughts on massively scalable gaussian processes. *arXiv preprint arXiv:1511.01870*, 2015.
- Andrew Gordon Wilson, Zhiting Hu, Ruslan Salakhutdinov, and Eric P Xing. Deep kernel learning. In *Artificial intelligence and statistics*, pages 370–378. PMLR, 2016b.
- Stephen J Wright. Numerical optimization, 2006.
- Kaichao You, Mingsheng Long, Jianmin Wang, and Michael I Jordan. How does learning rate decay help modern neural networks? *arXiv preprint arXiv:1908.01878*, 2019.
- Manzil Zaheer, Sashank Reddi, Devendra Sachan, Satyen Kale, and Sanjiv Kumar. Adaptive methods for nonconvex optimization. *Advances in neural information processing systems*, 31, 2018.